



```
In [10]: #@AUTHOR-SOUPARNO
```

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr

np.random.seed(42)
pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", "{:.3f}".format)
sns.set_style("whitegrid")
```

```
In [3]: df = pd.read_excel("LLM_Usage_Trends.xlsx")
```

```
In [4]: df.shape
```

```
Out[4]: (600, 30)
```

```
In [5]: df.head()
```

```
Out[5]:
```

	User_ID	Job_Role	Industry	Experience_Level	Years_of_Experience	Age
0	USR0001	Data Scientist	Information Technology	Mid-Level	5.400	
1	USR0002	Operations Manager	Manufacturing & Operations	Lead/Managerial	18.900	
2	USR0003	Marketing Manager	Marketing & Advertising	Senior-Level	6.800	
3	USR0004	Healthcare Professional	Healthcare	Mid-Level	5.400	
4	USR0005	Digital Marketer	Marketing & Advertising	Mid-Level	3.200	

```
In [6]: df.head(-80)
```

Out[6]:

	User_ID	Job_Role	Industry	Experience_Level	Years_of_Experience
0	USR0001	Data Scientist	Information Technology	Mid-Level	5.400
1	USR0002	Operations Manager	Manufacturing & Operations	Lead/Managerial	18.900
2	USR0003	Marketing Manager	Marketing & Advertising	Senior-Level	6.800
3	USR0004	Healthcare Professional	Healthcare	Mid-Level	5.400
4	USR0005	Digital Marketer	Marketing & Advertising	Mid-Level	3.200
...	...	...	...	...	...
515	USR0516	Customer Support Executive	BPO/ Customer Service	Entry-Level	0.800
516	USR0517	Software Engineer	Information Technology	Senior-Level	9.900
517	USR0518	UI/UX Designer	Information Technology	Lead/Managerial	14.900
518	USR0519	Student	Education	Mid-Level	5.700
519	USR0520	Research Analyst	Consulting	Mid-Level	5.800

520 rows × 30 columns

In [7]: `df.tail(-10)`

Out[7]:

	User_ID	Job_Role	Industry	Experience_Level	Years_of_Experience	Age
10	USR0011	UI/UX Designer	Information Technology	Senior-Level	6.600	
11	USR0012	HR Manager	Human Resources	Entry-Level	0.500	
12	USR0013	Content Writer	Media & Publishing	Senior-Level	8.400	
13	USR0014	Graphic Designer	Media & Publishing	Senior-Level	9.900	
14	USR0015	Student	Education	Mid-Level	3.300	
...	...	...	...	...	...	
595	USR0596	Business Analyst	Consulting	Mid-Level	2.200	
596	USR0597	Sales Executive	Sales & Retail	Senior-Level	12.000	
597	USR0598	Data Scientist	Information Technology	Senior-Level	9.700	
598	USR0599	Business Analyst	Consulting	Lead/Managerial	16.300	
599	USR0600	Recruiter	Human Resources	Senior-Level	6.700	

590 rows × 30 columns

In [8]:

```

num_cols = ["Years_of_Experience", "Monthly_Cost_USD", "Hours_Per_Week", "Months_Since_Adoption",
            "Tasks_Automated_Per_Week", "Productivity_Gain_Percent", "Time_Saved_Hours_Per_Week",
            "Accuracy_Satisfaction_Score_1to10", "Overall_Satisfaction_Score_1to10", "Trust_Level_1to10"]

cat_cols = ["Job_Role", "Industry", "Experience_Level", "Age_Group", "Country", "Company_Size",
            "Primary_LLM_Tool", "Primary_Model_Used", "Secondary_LLM_Tool", "Subscription_Type",
            "Usage_Frequency", "Primary_Use_Case", "Secondary_Use_Case", "Would_Recommend",
            "Main_Concern", "Team_Also_Uses_LLM", "Company_Provided_Training"]

print("Numerical columns:", num_cols)
print("Categorical columns:", cat_cols)

```

Numerical columns: ['Years_of_Experience', 'Monthly_Cost_USD', 'Hours_Per_Week', 'Months_Since_Adoption', 'Tasks_Automated_Per_Week', 'Productivity_Gain_Percent', 'Time_Saved_Hours_Per_Week', 'Accuracy_Satisfaction_Score_1to10', 'Overall_Satisfaction_Score_1to10', 'Trust_Level_1to10']

Categorical columns: ['Job_Role', 'Industry', 'Experience_Level', 'Age_Group', 'Country', 'Company_Size', 'Primary_LLM_Tool', 'Primary_Model_Used', 'Secondary_LLM_Tool', 'Subscription_Type', 'Usage_Frequency', 'Primary_Use_Case', 'Secondary_Use_Case', 'Would_Recommend', 'Main_Concern', 'Team_Also_Uses_LLM', 'Company_Provided_Training']

```
In [9]: print("Shape:", df.shape)
print("\nColumn names:\n", df.columns.tolist())
print("-"*50)
print("\nData types:")
print(df.dtypes)
print("-"*50)
print("\nUnique values per column:")
print(df.nunique())
```

Shape: (600, 30)

Column names:

```
['User_ID', 'Job_Role', 'Industry', 'Experience_Level', 'Years_of_Experience',  
'Age_Group', 'Country', 'Company_Size', 'Primary_LLM_Tool', 'Primary_Model_Used',  
'Secondary_LLM_Tool', 'Secondary_Model_Used', 'Subscription_Type', 'Monthly_Cost_USD',  
'Usage_Frequency', 'Hours_Per_Week', 'Adoption_Date', 'Months_Since_Adoption',  
'Primary_Use_Case', 'Secondary_Use_Case', 'Tasks_Automated_Per_Week',  
'Productivity_Gain_Percent', 'Time_Saved_Hours_Per_Week', 'Accuracy_Satisfaction_Score_1to10',  
'Overall_Satisfaction_Score_1to10', 'Trust_Level_1to10', 'Would_Recommend', 'Main_Concern',  
'Team_Also_Uses_LLM', 'Company_Provided_Training']
```

Data types:

User_ID	str
Job_Role	str
Industry	str
Experience_Level	str
Years_of_Experience	float64
Age_Group	str
Country	str
Company_Size	str
Primary_LLM_Tool	str
Primary_Model_Used	str
Secondary_LLM_Tool	str
Secondary_Model_Used	str
Subscription_Type	str
Monthly_Cost_USD	int64
Usage_Frequency	str
Hours_Per_Week	float64
Adoption_Date	str
Months_Since_Adoption	int64
Primary_Use_Case	str
Secondary_Use_Case	str
Tasks_Automated_Per_Week	int64
Productivity_Gain_Percent	int64
Time_Saved_Hours_Per_Week	float64
Accuracy_Satisfaction_Score_1to10	int64
Overall_Satisfaction_Score_1to10	int64
Trust_Level_1to10	int64
Would_Recommend	str
Main_Concern	str
Team_Also_Uses_LLM	str
Company_Provided_Training	str

dtype: object

Unique values per column:

User_ID	600
Job_Role	20
Industry	12
Experience_Level	4
Years_of_Experience	159

Age_Group	5
Country	10
Company_Size	4
Primary_LLM_Tool	11
Primary_Model_Used	33
Secondary_LLM_Tool	11
Secondary_Model_Used	33
Subscription_Type	3
Monthly_Cost_USD	8
Usage_Frequency	5
Hours_Per_Week	181
Adoption_Date	469
Months_Since_Adoption	43
Primary_Use_Case	10
Secondary_Use_Case	12
Tasks_Automated_Per_Week	31
Productivity_Gain_Percent	69
Time_Saved_Hours_Per_Week	166
Accuracy_Satisfaction_Score_1to10	9
Overall_Satisfaction_Score_1to10	9
Trust_Level_1to10	10
Would_Recommend	2
Main_Concern	5
Team_Also_Uses_LLM	2
Company_Provided_Training	2
dtype:	int64

```
In [11]: print("First pass info:")
          print(df.info())
```

First pass info:

<class 'pandas.DataFrame'>

RangeIndex: 600 entries, 0 to 599

Data columns (total 30 columns):

#	Column	Non-Null Count	Dtype
0	User_ID	600 non-null	str
1	Job_Role	600 non-null	str
2	Industry	600 non-null	str
3	Experience_Level	600 non-null	str
4	Years_of_Experience	600 non-null	float64
5	Age_Group	600 non-null	str
6	Country	600 non-null	str
7	Company_Size	600 non-null	str
8	Primary_LLM_Tool	600 non-null	str
9	Primary_Model_Used	600 non-null	str
10	Secondary_LLM_Tool	332 non-null	str
11	Secondary_Model_Used	332 non-null	str
12	Subscription_Type	600 non-null	str
13	Monthly_Cost_USD	600 non-null	int64
14	Usage_Frequency	600 non-null	str
15	Hours_Per_Week	600 non-null	float64
16	Adoption_Date	600 non-null	str
17	Months_Since_Adoption	600 non-null	int64
18	Primary_Use_Case	600 non-null	str
19	Secondary_Use_Case	600 non-null	str
20	Tasks_Automated_Per_Week	600 non-null	int64
21	Productivity_Gain_Percent	600 non-null	int64
22	Time_Saved_Hours_Per_Week	600 non-null	float64
23	Accuracy_Satisfaction_Score_1to10	600 non-null	int64
24	Overall_Satisfaction_Score_1to10	600 non-null	int64
25	Trust_Level_1to10	600 non-null	int64
26	Would_Recommend	600 non-null	str
27	Main_Concern	511 non-null	str
28	Team_Also_Uses_LLM	600 non-null	str
29	Company_Provided_Training	600 non-null	str

dtypes: float64(3), int64(7), str(20)

memory usage: 140.8 KB

None

```
In [13]: # descriptive statistics
df[num_cols].describe()
```

Out[13]:	Years_of_Experience	Monthly_Cost_USD	Hours_Per_Week	Months_Since_
count	600.000	600.000	600.000	
mean	5.953	13.670	7.662	
std	4.941	12.883	5.776	
min	0.000	0.000	0.100	
25%	1.900	0.000	2.200	
50%	4.800	20.000	6.600	
75%	9.025	22.000	11.525	
max	19.900	40.000	20.000	

In [16]: *#Missing values*

```
In [17]: missing_count = df.isna().sum().sort_values(ascending=False)
missing_percent = (df.isna().mean() * 100).sort_values(ascending=False)
```

```
In [18]: missing_summary = pd.DataFrame({
    "missing_count": missing_count,
    "missing_percent": missing_percent.round(2)
})

print("Missing values summary:")
missing_summary
```

Missing values summary:

Out[18]:

	missing_count	missing_percent
Secondary_LLM_Tool	268	44.670
Secondary_Model_Used	268	44.670
Main_Concern	89	14.830
Industry	0	0.000
User_ID	0	0.000
Experience_Level	0	0.000
Age_Group	0	0.000
Years_of_Experience	0	0.000
Country	0	0.000
Company_Size	0	0.000
Primary_Model_Used	0	0.000
Primary_LLM_Tool	0	0.000
Subscription_Type	0	0.000
Job_Role	0	0.000
Monthly_Cost_USD	0	0.000
Usage_Frequency	0	0.000
Adoption_Date	0	0.000
Hours_Per_Week	0	0.000
Primary_Use_Case	0	0.000
Secondary_Use_Case	0	0.000
Tasks_Automated_Per_Week	0	0.000
Months_Since_Adoption	0	0.000
Productivity_Gain_Percent	0	0.000
Time_Saved_Hours_Per_Week	0	0.000
Overall_Satisfaction_Score_1to10	0	0.000
Accuracy_Satisfaction_Score_1to10	0	0.000
Trust_Level_1to10	0	0.000
Would_Recommend	0	0.000
Team_Also_Uses_LLM	0	0.000
Company_Provided_Training	0	0.000

```
In [19]: df.isnull().sum()
```

```
Out[19]: User_ID          0
         Job_Role        0
         Industry        0
         Experience_Level 0
         Years_of_Experience 0
         Age_Group       0
         Country         0
         Company_Size    0
         Primary_LLM_Tool 0
         Primary_Model_Used 0
         Secondary_LLM_Tool 268
         Secondary_Model_Used 268
         Subscription_Type 0
         Monthly_Cost_USD 0
         Usage_Frequency 0
         Hours_Per_Week   0
         Adoption_Date    0
         Months_Since_Adoption 0
         Primary_Use_Case 0
         Secondary_Use_Case 0
         Tasks_Automated_Per_Week 0
         Productivity_Gain_Percent 0
         Time_Saved_Hours_Per_Week 0
         Accuracy_Satisfaction_Score_1to10 0
         Overall_Satisfaction_Score_1to10 0
         Trust_Level_1to10 0
         Would_Recommend 0
         Main_Concern     89
         Team_Also_Uses_LLM 0
         Company_Provided_Training 0
         dtype: int64
```

```
In [20]: #DUPLICATES
```

```
In [21]: duplicate_mask = df.duplicated()
         num_duplicates = duplicate_mask.sum()

         print("Number of duplicate rows:", num_duplicates)
```

Number of duplicate rows: 0

```
In [22]: # User_ID should always be unique
         print("Duplicate User_IDs:", df["User_ID"].duplicated().sum())
```

Duplicate User_IDs: 0

```
In [23]: expected_types = {
         "Years_of_Experience": "float64",
         "Monthly_Cost_USD": "int64",
         "Hours_Per_Week": "float64",
         "Months_Since_Adoption": "int64",
         "Job_Role": "object",
```

```
"Primary_LLM_Tool": "object",  
"Adoption_Date": "object"  
}
```

```
In [24]: print("Data type validation:")  
for col, expected in expected_types.items():  
    actual = str(df[col].dtype)  
    status = "OK" if actual == expected else "MISMATCH"  
    print(f"{col}: expected={expected}, actual={actual} -> {status}")
```

Data type validation:

Years_of_Experience: expected=float64, actual=float64 -> OK

Monthly_Cost_USD: expected=int64, actual=int64 -> OK

Hours_Per_Week: expected=float64, actual=float64 -> OK

Months_Since_Adoption: expected=int64, actual=int64 -> OK

Job_Role: expected=object, actual=str -> MISMATCH

Primary_LLM_Tool: expected=object, actual=str -> MISMATCH

Adoption_Date: expected=object, actual=str -> MISMATCH

```
In [25]: # convert Adoption_Date to a real datetime for time-based analysis later  
df["Adoption_Date"] = pd.to_datetime(df["Adoption_Date"])  
df["Adoption_Date"].dtype
```

```
Out[25]: dtype('<M8[us]')
```

```
In [26]: #Numerical features
```

```
In [27]: print("Mean:")  
print(df[num_cols].mean())  
print("-"*50)  
print("Median:")  
print(df[num_cols].median())  
print("-"*50)  
print("Mode:")  
print(df[num_cols].mode().iloc[0])
```

```

Mean:
Years_of_Experience          5.953
Monthly_Cost_USD            13.670
Hours_Per_Week               7.662
Months_Since_Adoption       23.365
Tasks_Automated_Per_Week    9.862
Productivity_Gain_Percent   29.970
Time_Saved_Hours_Per_Week   7.728
Accuracy_Satisfaction_Score_1to10 6.412
Overall_Satisfaction_Score_1to10 6.225
Trust_Level_1to10           4.748
dtype: float64
-----
Median:
Years_of_Experience          4.800
Monthly_Cost_USD            20.000
Hours_Per_Week               6.600
Months_Since_Adoption       24.000
Tasks_Automated_Per_Week    8.000
Productivity_Gain_Percent   28.000
Time_Saved_Hours_Per_Week   6.900
Accuracy_Satisfaction_Score_1to10 6.000
Overall_Satisfaction_Score_1to10 6.000
Trust_Level_1to10           5.000
dtype: float64
-----
Mode:
Years_of_Experience          0.300
Monthly_Cost_USD            0.000
Hours_Per_Week               1.000
Months_Since_Adoption       22.000
Tasks_Automated_Per_Week    1.000
Productivity_Gain_Percent   18.000
Time_Saved_Hours_Per_Week   6.900
Accuracy_Satisfaction_Score_1to10 7.000
Overall_Satisfaction_Score_1to10 7.000
Trust_Level_1to10           5.000
Name: 0, dtype: float64

```

```
In [28]: #Spread
```

```
In [29]: print("Variance:")
print(df[num_cols].var())
print("-"*50)
print("Standard Deviation:")
print(df[num_cols].std())
```

```

Variance:
Years_of_Experience      24.415
Monthly_Cost_USD        165.984
Hours_Per_Week           33.362
Months_Since_Adoption    145.581
Tasks_Automated_Per_Week  59.769
Productivity_Gain_Percent 242.937
Time_Saved_Hours_Per_Week 20.259
Accuracy_Satisfaction_Score_1to10 2.583
Overall_Satisfaction_Score_1to10 3.567
Trust_Level_1to10        4.112
dtype: float64

```

```

-----
Standard Deviation:
Years_of_Experience      4.941
Monthly_Cost_USD        12.883
Hours_Per_Week           5.776
Months_Since_Adoption    12.066
Tasks_Automated_Per_Week  7.731
Productivity_Gain_Percent 15.586
Time_Saved_Hours_Per_Week 4.501
Accuracy_Satisfaction_Score_1to10 1.607
Overall_Satisfaction_Score_1to10 1.889
Trust_Level_1to10        2.028
dtype: float64

```

```
In [30]: # Skewness & kurtosis
```

```
In [31]: print("Skewness:")
print(df[num_cols].skew())
print("\nKurtosis:")
print(df[num_cols].kurtosis())
```

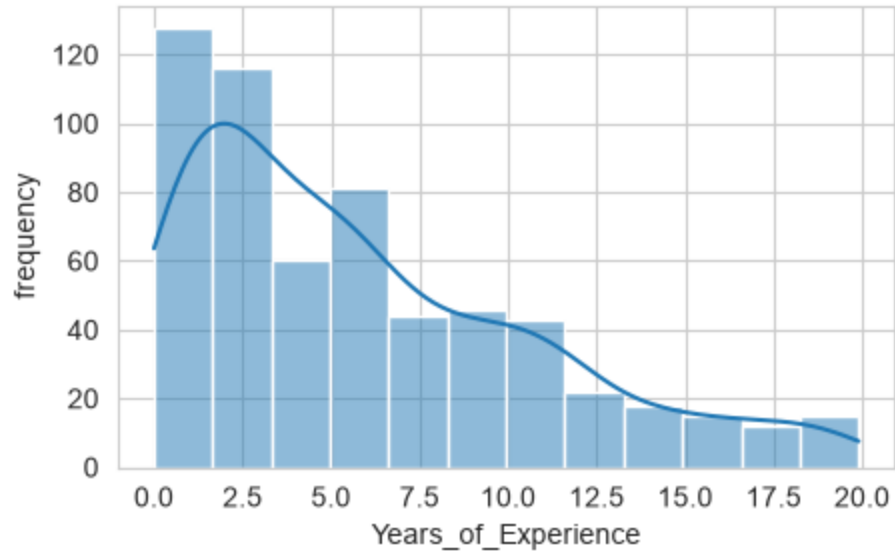
```
Skewness:
Years_of_Experience          0.933
Monthly_Cost_USD            0.208
Hours_Per_Week               0.530
Months_Since_Adoption       -0.072
Tasks_Automated_Per_Week    0.664
Productivity_Gain_Percent    0.388
Time_Saved_Hours_Per_Week   0.702
Accuracy_Satisfaction_Score_1to10 -0.049
Overall_Satisfaction_Score_1to10 -0.145
Trust_Level_1to10           0.156
dtype: float64
```

```
Kurtosis:
Years_of_Experience          0.093
Monthly_Cost_USD            -1.280
Hours_Per_Week               -0.910
Months_Since_Adoption       -1.143
Tasks_Automated_Per_Week    -0.571
Productivity_Gain_Percent    -0.533
Time_Saved_Hours_Per_Week   0.026
Accuracy_Satisfaction_Score_1to10 -0.202
Overall_Satisfaction_Score_1to10 -0.490
Trust_Level_1to10           -0.348
dtype: float64
```

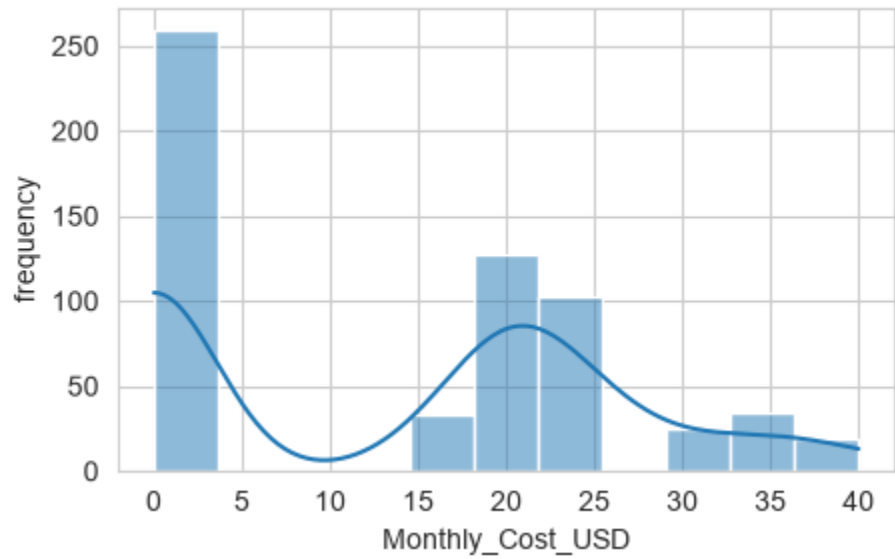
```
In [32]: #Histograms
```

```
In [33]: for col in num_cols:
          plt.figure(figsize=(5, 3))
          sns.histplot(df[col], kde=True)
          plt.title(f"Histogram: {col}")
          plt.xlabel(col)
          plt.ylabel("frequency")
          plt.show()
```

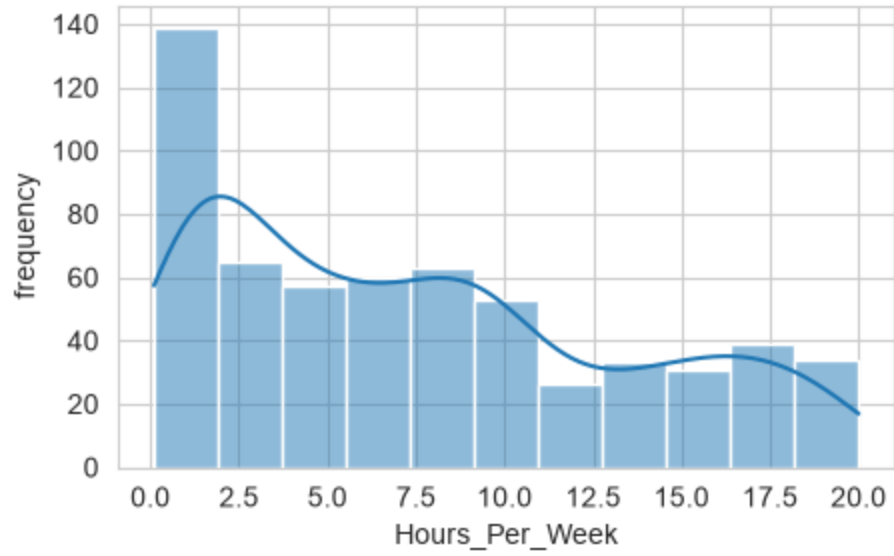
Histogram: Years_of_Experience



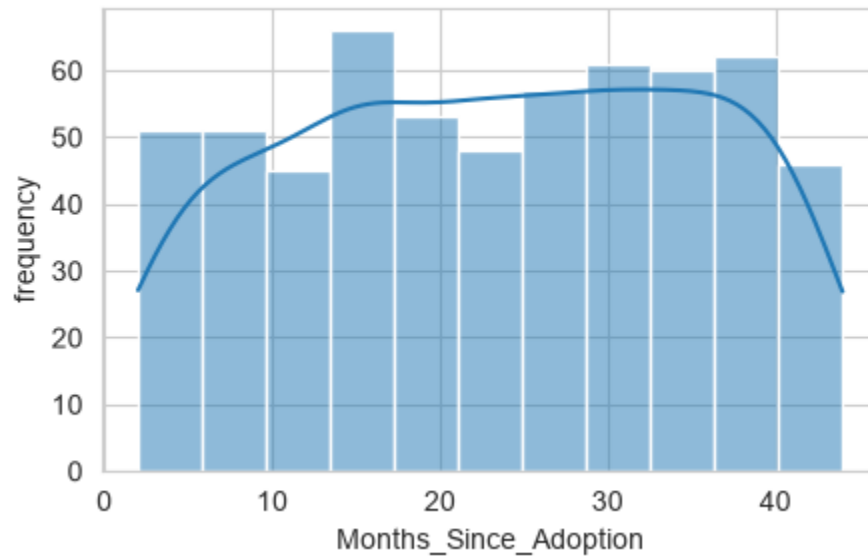
Histogram: Monthly_Cost_USD



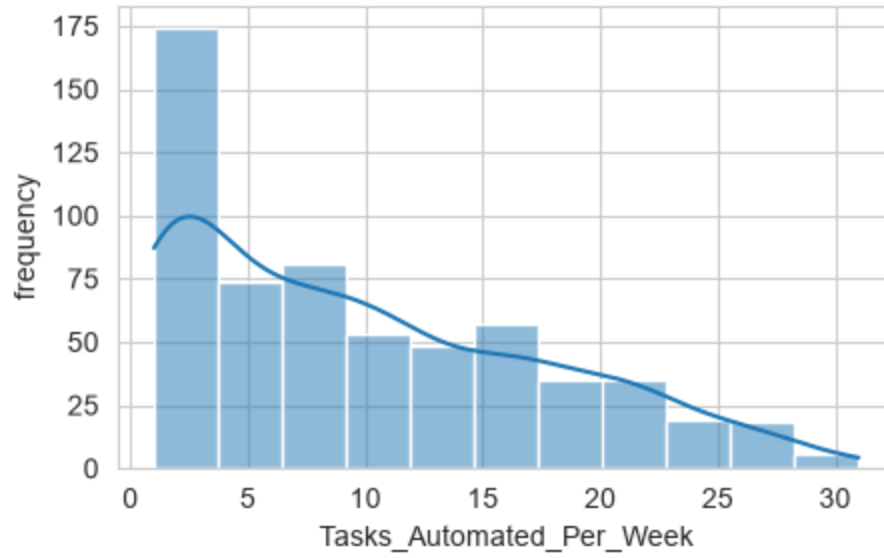
Histogram: Hours_Per_Week



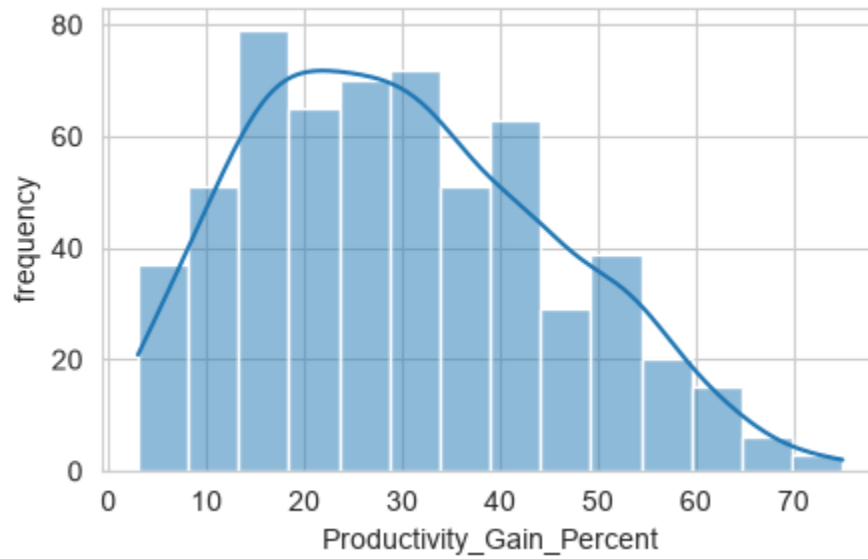
Histogram: Months_Since_Adoption

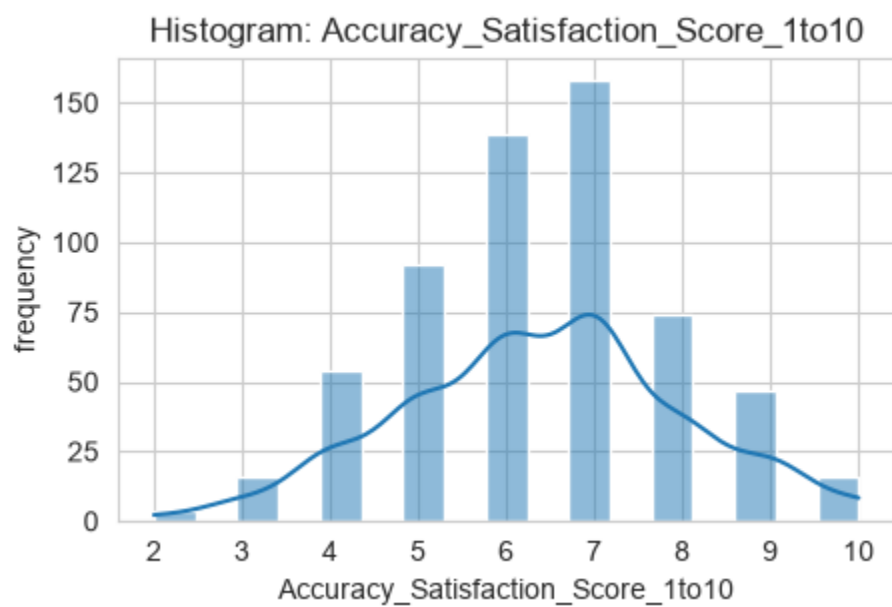
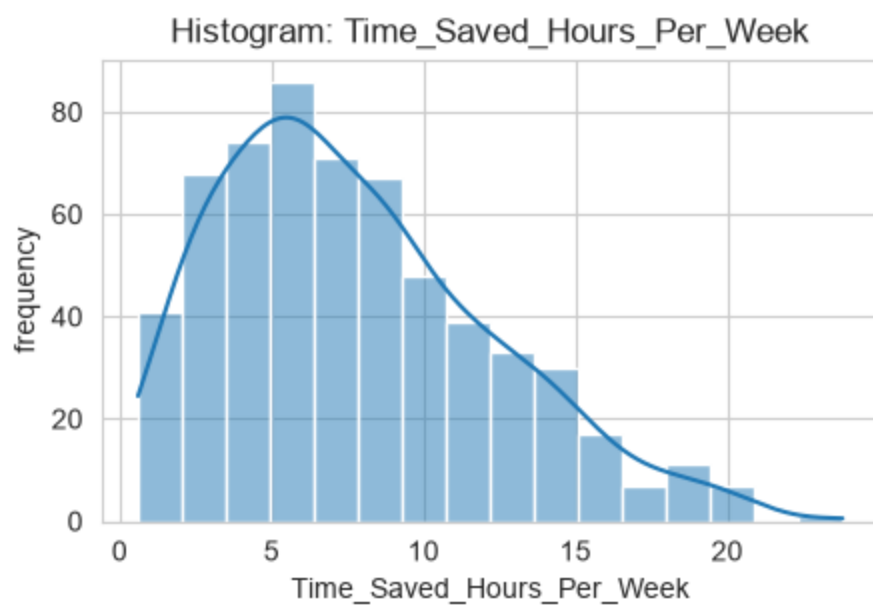


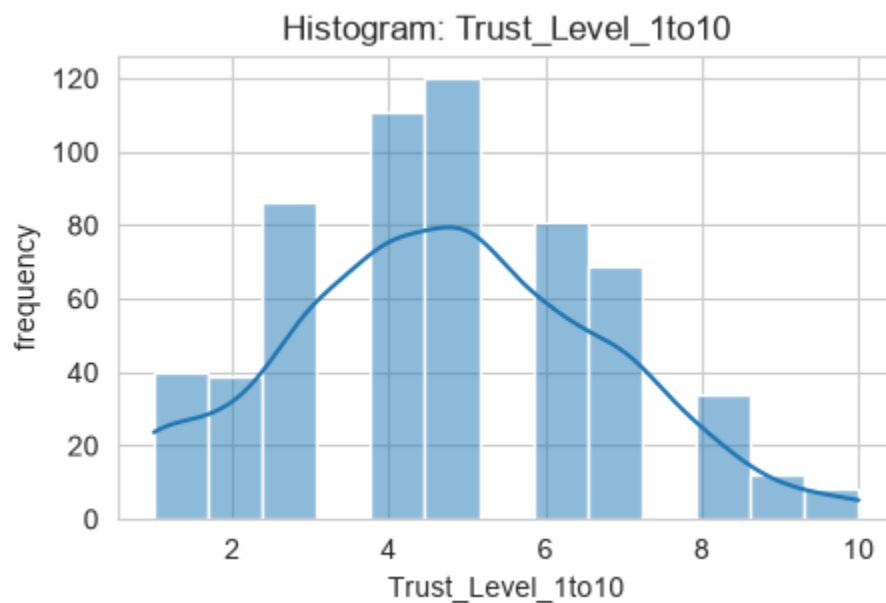
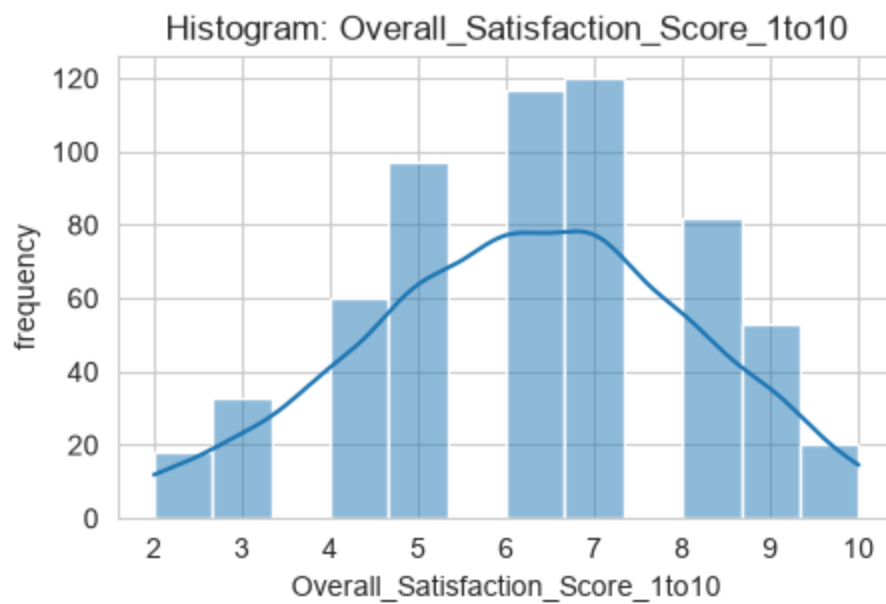
Histogram: Tasks_Automated_Per_Week



Histogram: Productivity_Gain_Percent



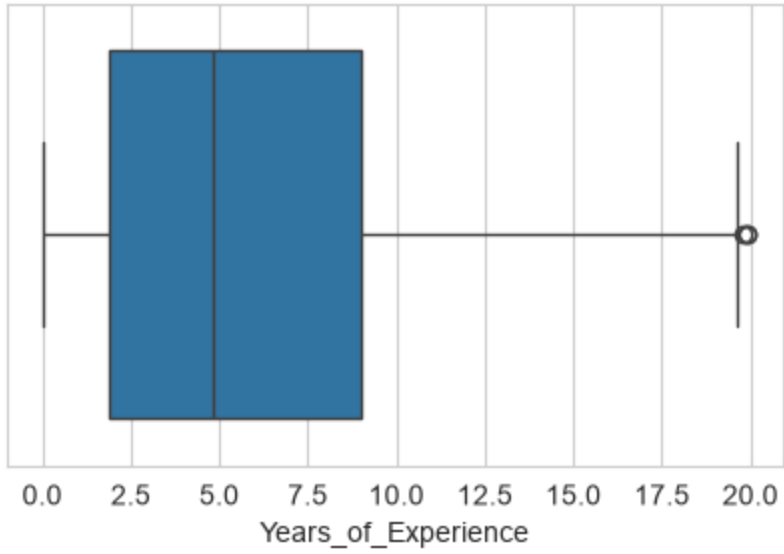




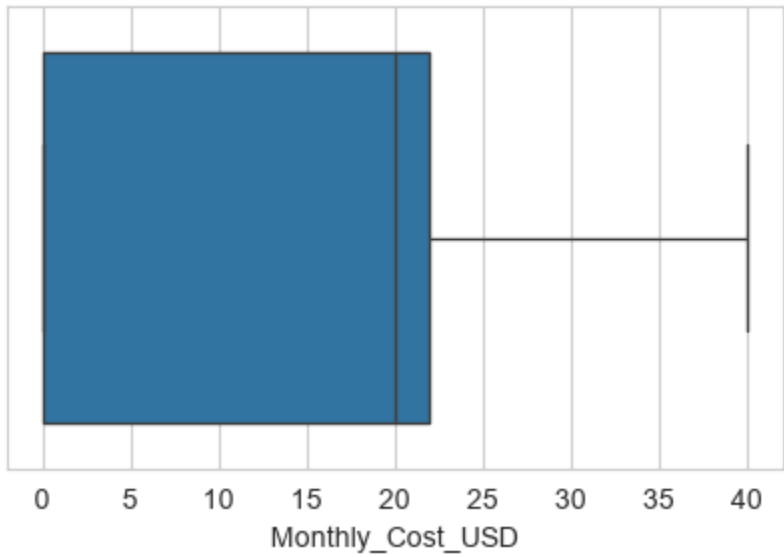
```
In [34]: # Boxplots (outlier check)
```

```
In [35]: for col in num_cols:
plt.figure(figsize=(5, 3))
sns.boxplot(x=df[col])
plt.title(f"Boxplot: {col}")
plt.xlabel(col)
plt.show()
```

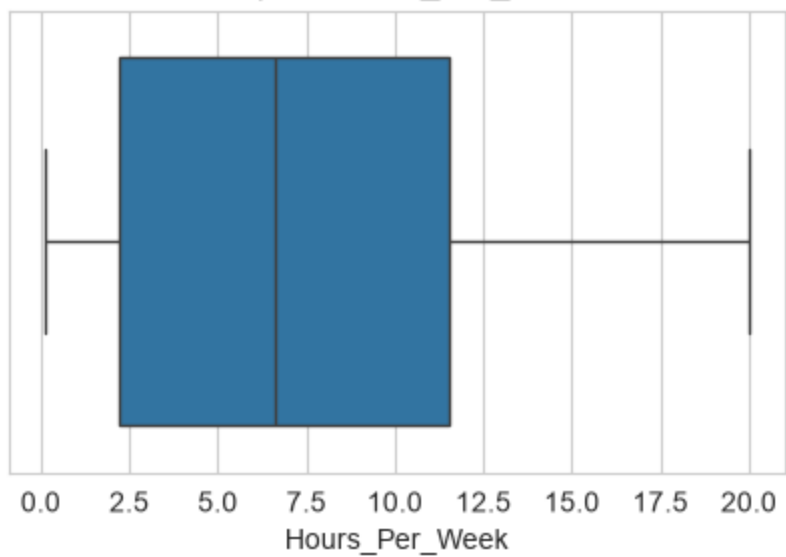
Boxplot: Years_of_Experience



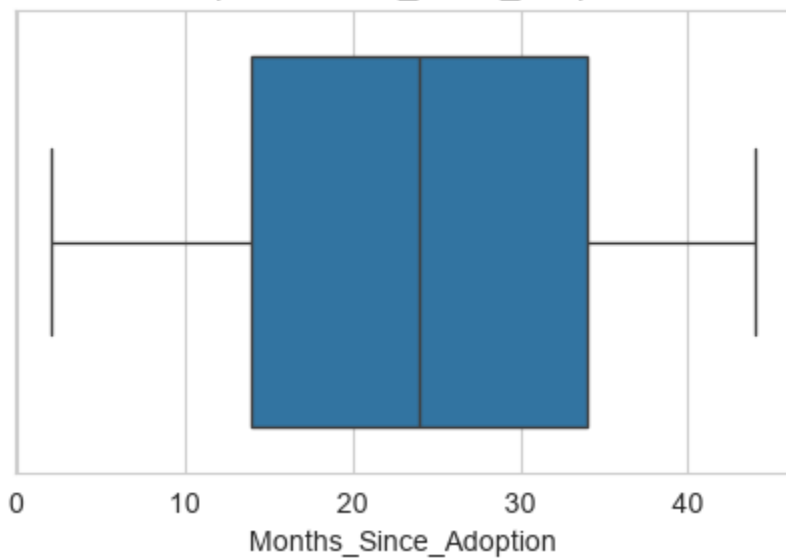
Boxplot: Monthly_Cost_USD



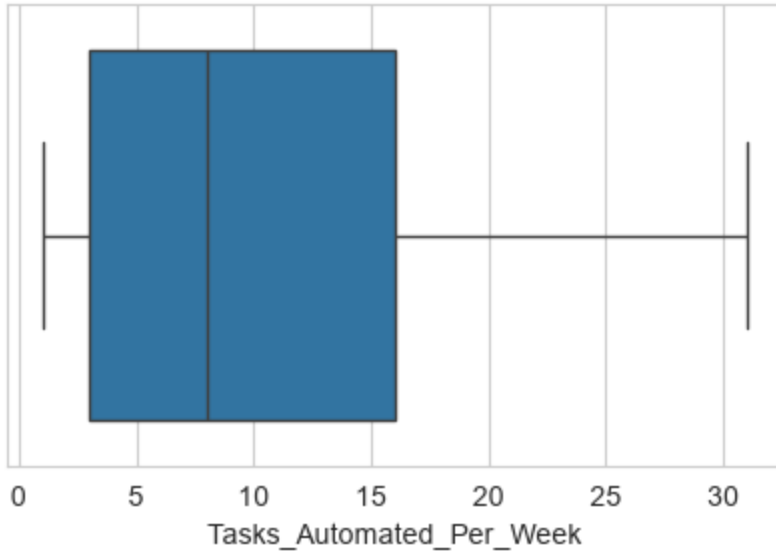
Boxplot: Hours_Per_Week



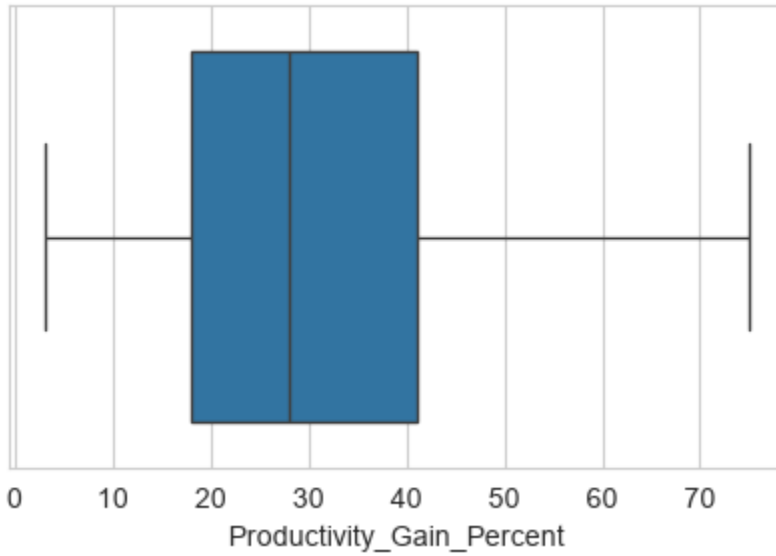
Boxplot: Months_Since_Adoption



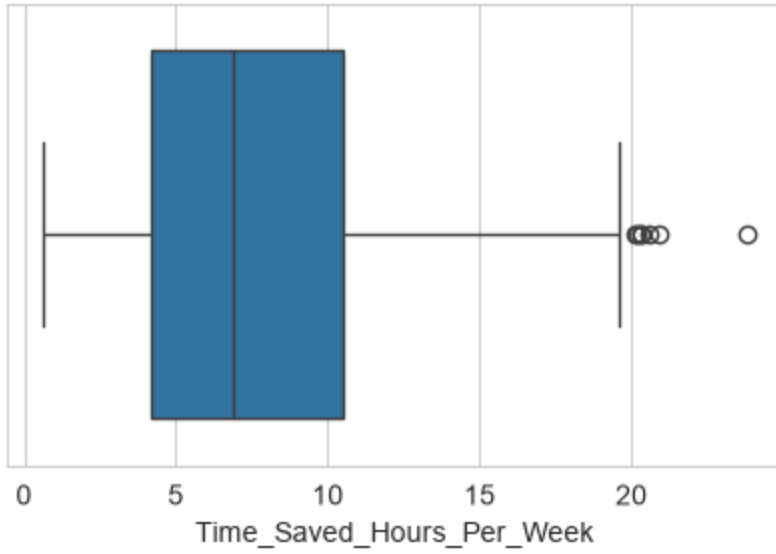
Boxplot: Tasks_Automated_Per_Week



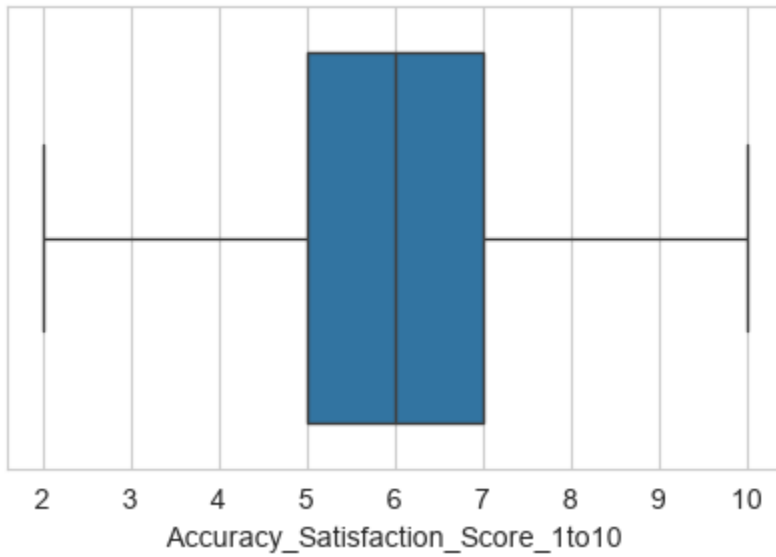
Boxplot: Productivity_Gain_Percent

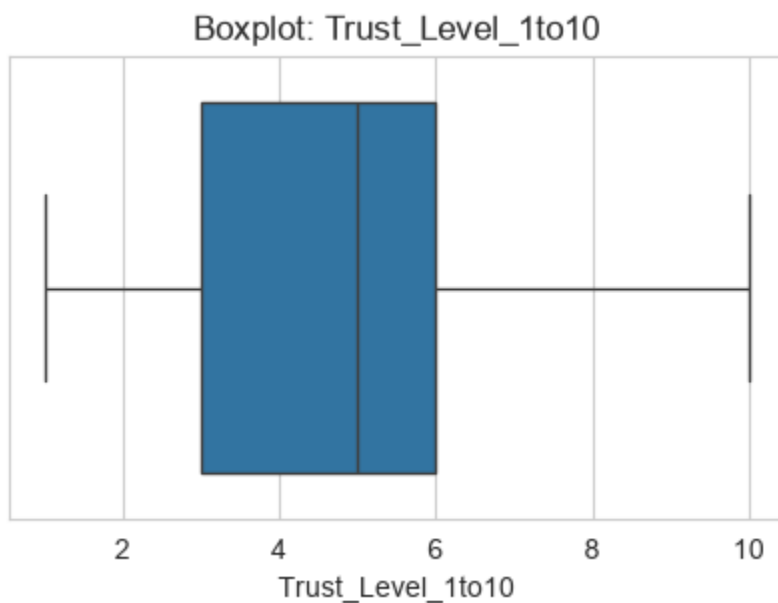
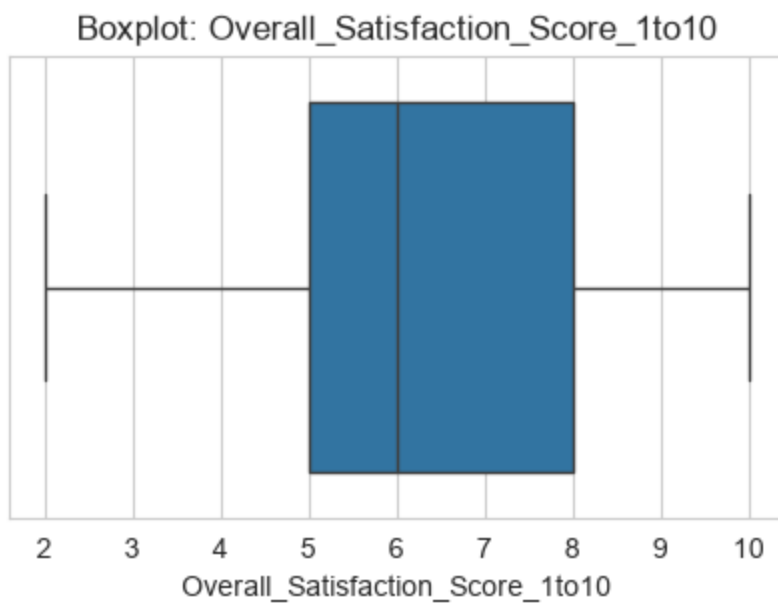


Boxplot: Time_Saved_Hours_Per_Week



Boxplot: Accuracy_Satisfaction_Score_1to10





In [36]: *#Frequency tables*

```
In [37]: for col in cat_cols:
          print(f"Frequency table for {col}")
          print(df[col].value_counts())
          print(f"Proportion for {col}")
          print(df[col].value_counts(normalize=True).round(3))
          print("-"*50)
```

Frequency table for Job_Role

Job_Role	
Digital Marketer	39
UI/UX Designer	39
Healthcare Professional	37
Graphic Designer	36
Research Analyst	35
Legal Associate	34
Content Writer	32
Customer Support Executive	31
Sales Executive	30
Data Scientist	29
Marketing Manager	29
HR Manager	29
Financial Analyst	28
Operations Manager	27
Business Analyst	27
Software Engineer	26
Product Manager	25
Student	24
Teacher/Educator	23
Recruiter	20

Name: count, dtype: int64

Proportion for Job_Role

Job_Role	
Digital Marketer	0.065
UI/UX Designer	0.065
Healthcare Professional	0.062
Graphic Designer	0.060
Research Analyst	0.058
Legal Associate	0.057
Content Writer	0.053
Customer Support Executive	0.052
Sales Executive	0.050
Data Scientist	0.048
Marketing Manager	0.048
HR Manager	0.048
Financial Analyst	0.047
Operations Manager	0.045
Business Analyst	0.045
Software Engineer	0.043
Product Manager	0.042
Student	0.040
Teacher/Educator	0.038
Recruiter	0.033

Name: proportion, dtype: float64

Frequency table for Industry

Industry	
Information Technology	119
Marketing & Advertising	68
Media & Publishing	68
Consulting	62
Human Resources	49

Education	47
Healthcare	37
Legal Services	34
BP0/Customer Service	31
Sales & Retail	30
Finance & Banking	28
Manufacturing & Operations	27

Name: count, dtype: int64

Proportion for Industry

Industry	
Information Technology	0.198
Marketing & Advertising	0.113
Media & Publishing	0.113
Consulting	0.103
Human Resources	0.082
Education	0.078
Healthcare	0.062
Legal Services	0.057
BP0/Customer Service	0.052
Sales & Retail	0.050
Finance & Banking	0.047
Manufacturing & Operations	0.045

Name: proportion, dtype: float64

Frequency table for Experience_Level

Experience_Level

Mid-Level	208
Entry-Level	159
Senior-Level	144
Lead/Managerial	89

Name: count, dtype: int64

Proportion for Experience_Level

Experience_Level

Mid-Level	0.347
Entry-Level	0.265
Senior-Level	0.240
Lead/Managerial	0.148

Name: proportion, dtype: float64

Frequency table for Age_Group

Age_Group

25-34	214
35-44	169
18-24	109
45-54	79
55+	29

Name: count, dtype: int64

Proportion for Age_Group

Age_Group

25-34	0.357
35-44	0.282
18-24	0.182
45-54	0.132
55+	0.048

Name: proportion, dtype: float64

Frequency table for Country

Country

India	203
United States	135
United Kingdom	55
Canada	45
Australia	33
Singapore	30
Germany	30
Brazil	28
France	26
UAE	15

Name: count, dtype: int64

Proportion for Country

Country

India	0.338
United States	0.225
United Kingdom	0.092
Canada	0.075
Australia	0.055
Singapore	0.050
Germany	0.050
Brazil	0.047
France	0.043
UAE	0.025

Name: proportion, dtype: float64

Frequency table for Company_Size

Company_Size

Mid-size (201-1000)	173
Small (51-200)	163
Large (1000+)	142
Startup (1-50)	122

Name: count, dtype: int64

Proportion for Company_Size

Company_Size

Mid-size (201-1000)	0.288
Small (51-200)	0.272
Large (1000+)	0.237
Startup (1-50)	0.203

Name: proportion, dtype: float64

Frequency table for Primary_LLM_Tool

Primary_LLM_Tool

ChatGPT	181
Gemini	99
Claude	81
Perplexity	54
Microsoft Copilot	53
Mistral (Le Chat)	31
Meta AI (Llama)	28
Qwen	20

Grok	19
Cohere (Command)	18
DeepSeek	16
Name: count, dtype: int64	
Proportion for Primary_LLM_Tool	
Primary_LLM_Tool	
ChatGPT	0.302
Gemini	0.165
Claude	0.135
Perplexity	0.090
Microsoft Copilot	0.088
Mistral (Le Chat)	0.052
Meta AI (Llama)	0.047
Qwen	0.033
Grok	0.032
Cohere (Command)	0.030
DeepSeek	0.027
Name: proportion, dtype: float64	

Frequency table for Primary_Model_Used

Primary_Model_Used	
GPT-5.5 Instant	82
GPT-5.6 Sol (Plus)	71
Gemini 3.5 Flash	40
Gemini 3.1 Pro	38
Claude Haiku 4.5	33
GPT-5.6 Sol (Business)	28
Claude Sonnet 5	28
Copilot (GPT-5.5 based)	22
Gemini 3.1 Pro (Workspace)	21
Copilot Pro	21
Perplexity (Sonar Auto)	20
Claude Opus 4.8	20
Perplexity Pro (Sonar Pro)	19
Llama 4 Scout	19
Perplexity Enterprise Pro	15
Mistral Large 3 (Le Chat Pro)	15
Qwen3 32B	12
Microsoft 365 Copilot	10
Mistral Small 3.1	10
Grok 4.5 (SuperGrok)	9
Grok 4 Mini	8
Command A	8
Llama 4 Maverick	8
DeepSeek V4-Pro (Expert Mode)	7
Command R	7
DeepSeek V4-Flash	6
Qwen3.7 Max	6
Mistral Large 3 (Le Chat Enterprise)	6
DeepSeek V4-Pro (Team API)	3
Command A (Enterprise)	3
Grok 4 Heavy (SuperGrok Heavy)	2
Qwen3.7 Max (Enterprise)	2
Llama 4 Maverick (Enterprise API)	1

```

Name: count, dtype: int64
Proportion for Primary_Model_Used
Primary_Model_Used
GPT-5.5 Instant          0.137
GPT-5.6 Sol (Plus)      0.118
Gemini 3.5 Flash         0.067
Gemini 3.1 Pro           0.063
Claude Haiku 4.5         0.055
GPT-5.6 Sol (Business)  0.047
Claude Sonnet 5          0.047
Copilot (GPT-5.5 based)  0.037
Gemini 3.1 Pro (Workspace) 0.035
Copilot Pro              0.035
Perplexity (Sonar Auto)  0.033
Claude Opus 4.8          0.033
Perplexity Pro (Sonar Pro) 0.032
Llama 4 Scout            0.032
Perplexity Enterprise Pro 0.025
Mistral Large 3 (Le Chat Pro) 0.025
Qwen3 32B                0.020
Microsoft 365 Copilot    0.017
Mistral Small 3.1        0.017
Grok 4.5 (SuperGrok)     0.015
Grok 4 Mini              0.013
Command A                0.013
Llama 4 Maverick          0.013
DeepSeek V4-Pro (Expert Mode) 0.012
Command R                0.012
DeepSeek V4-Flash        0.010
Qwen3.7 Max              0.010
Mistral Large 3 (Le Chat Enterprise) 0.010
DeepSeek V4-Pro (Team API) 0.005
Command A (Enterprise)   0.005
Grok 4 Heavy (SuperGrok Heavy) 0.003
Qwen3.7 Max (Enterprise) 0.003
Llama 4 Maverick (Enterprise API) 0.002
Name: proportion, dtype: float64

```

Frequency table for Secondary_LLM_Tool

```

Secondary_LLM_Tool
Microsoft Copilot    39
Qwen                 36
Mistral (Le Chat)    33
DeepSeek             32
Claude              32
Perplexity           31
Meta AI (Llama)      30
Gemini               28
Cohere (Command)     27
Grok                 26
ChatGPT              18

```

```

Name: count, dtype: int64
Proportion for Secondary_LLM_Tool
Secondary_LLM_Tool

```

Microsoft Copilot	0.117
Qwen	0.108
Mistral (Le Chat)	0.099
DeepSeek	0.096
Claude	0.096
Perplexity	0.093
Meta AI (Llama)	0.090
Gemini	0.084
Cohere (Command)	0.081
Grok	0.078
ChatGPT	0.054

Name: proportion, dtype: float64

Frequency table for Subscription_Type

Subscription_Type	
Free	259
Paid (Individual)	230
Enterprise/Team	111

Name: count, dtype: int64

Proportion for Subscription_Type

Subscription_Type	
Free	0.432
Paid (Individual)	0.383
Enterprise/Team	0.185

Name: proportion, dtype: float64

Frequency table for Usage_Frequency

Usage_Frequency	
Daily	214
Several times a week	169
Weekly	116
Monthly	67
Rarely	34

Name: count, dtype: int64

Proportion for Usage_Frequency

Usage_Frequency	
Daily	0.357
Several times a week	0.282
Weekly	0.193
Monthly	0.112
Rarely	0.057

Name: proportion, dtype: float64

Frequency table for Primary_Use_Case

Primary_Use_Case	
Research & Summarization	106
Content Writing	100
Data Analysis	84
Email/Document Drafting	79
Image/Design Generation	75
Learning/Studying	47
Customer Query Resolution	31
Meeting Notes & Summaries	27
Coding & Debugging	26

```

Brainstorming & Ideation      25
Name: count, dtype: int64
Proportion for Primary_Use_Case
Primary_Use_Case
Research & Summarization      0.177
Content Writing                0.167
Data Analysis                 0.140
Email/Document Drafting       0.132
Image/Design Generation       0.125
Learning/Studying             0.078
Customer Query Resolution     0.052
Meeting Notes & Summaries     0.045
Coding & Debugging            0.043
Brainstorming & Ideation      0.042
Name: proportion, dtype: float64

```

Frequency table for Secondary_Use_Case

```

Secondary_Use_Case
Translation                  67
Learning/Studying            62
Coding & Debugging           54
Brainstorming & Ideation     53
Customer Query Resolution    52
Image/Design Generation      49
Research & Summarization     47
Data Analysis                46
Content Writing              46
Meeting Notes & Summaries    45
Email/Document Drafting      42
Presentation Building         37

```

Name: count, dtype: int64

Proportion for Secondary_Use_Case

```

Secondary_Use_Case
Translation                  0.112
Learning/Studying            0.103
Coding & Debugging           0.090
Brainstorming & Ideation     0.088
Customer Query Resolution    0.087
Image/Design Generation      0.082
Research & Summarization     0.078
Data Analysis                0.077
Content Writing              0.077
Meeting Notes & Summaries    0.075
Email/Document Drafting      0.070
Presentation Building         0.062

```

Name: proportion, dtype: float64

Frequency table for Would_Recommend

Would_Recommend

Yes 450

No 150

Name: count, dtype: int64

Proportion for Would_Recommend

Would_Recommend

```

Yes    0.750
No     0.250
Name: proportion, dtype: float64
-----
Frequency table for Main_Concern
Main_Concern
Accuracy of responses    184
Data Privacy             104
Subscription Cost        94
Job Security             81
Bias in Output           48
Name: count, dtype: int64
Proportion for Main_Concern
Main_Concern
Accuracy of responses    0.360
Data Privacy            0.204
Subscription Cost       0.184
Job Security            0.159
Bias in Output          0.094
Name: proportion, dtype: float64
-----
Frequency table for Team_Also_Uses_LLM
Team_Also_Uses_LLM
Yes    391
No     209
Name: count, dtype: int64
Proportion for Team_Also_Uses_LLM
Team_Also_Uses_LLM
Yes    0.652
No     0.348
Name: proportion, dtype: float64
-----
Frequency table for Company_Provided_Training
Company_Provided_Training
No     363
Yes    237
Name: count, dtype: int64
Proportion for Company_Provided_Training
Company_Provided_Training
No     0.605
Yes    0.395
Name: proportion, dtype: float64
-----

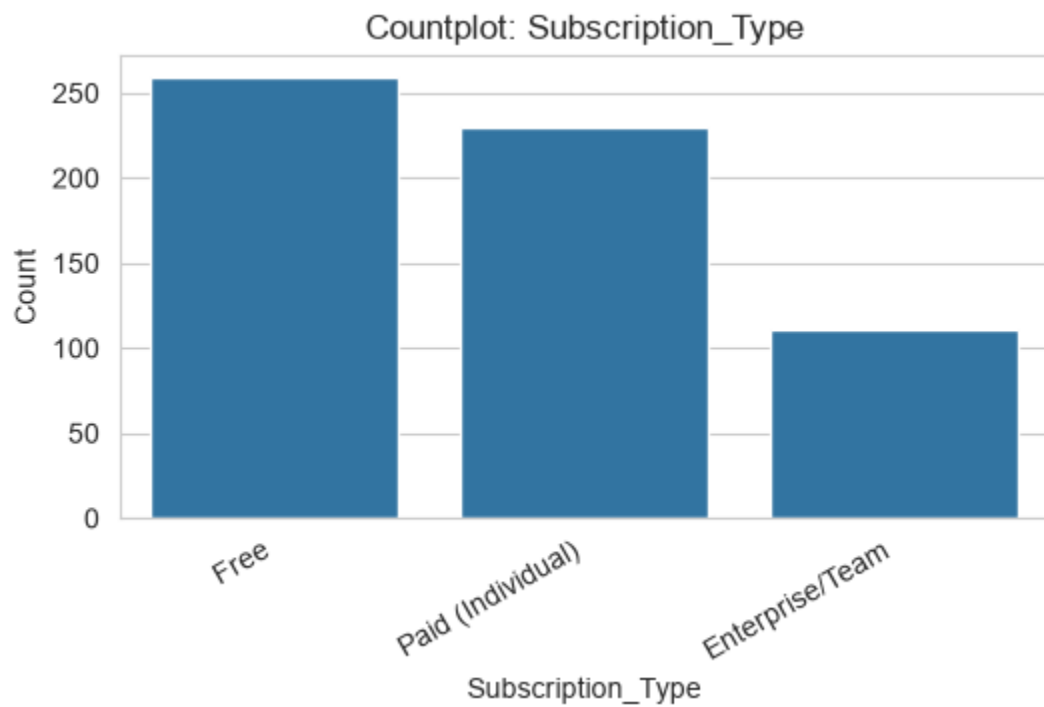
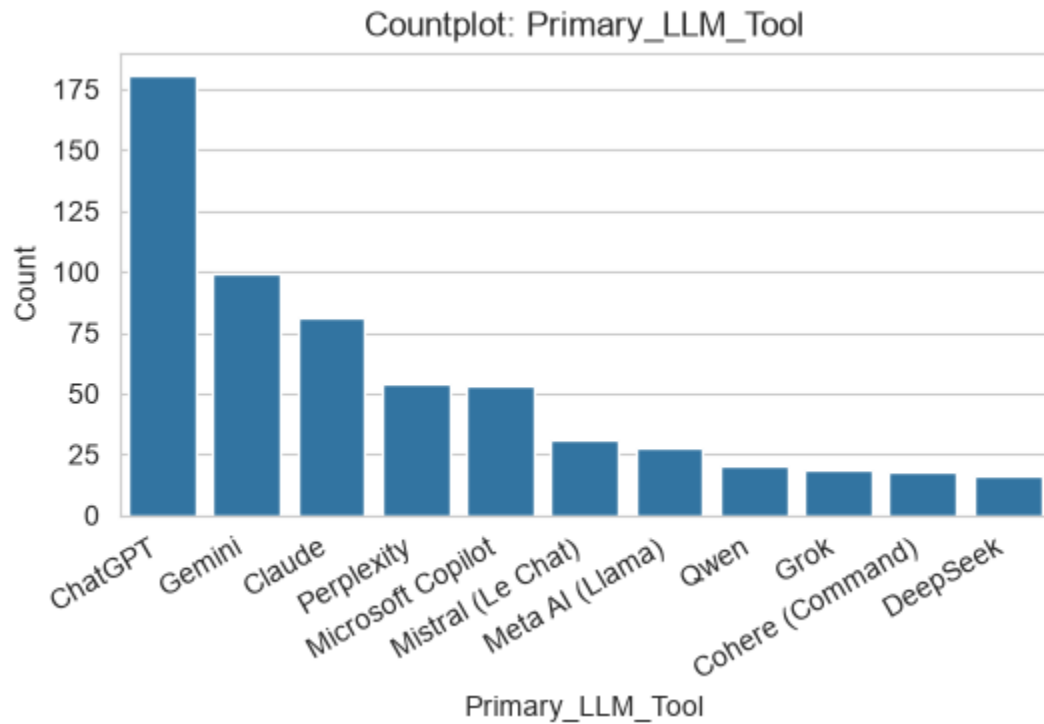
```

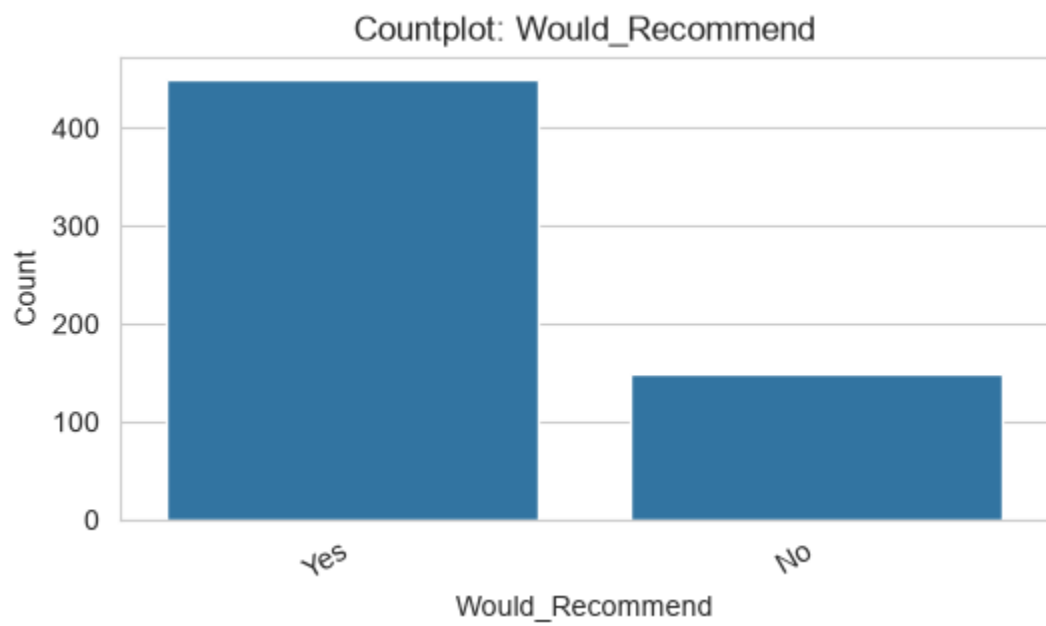
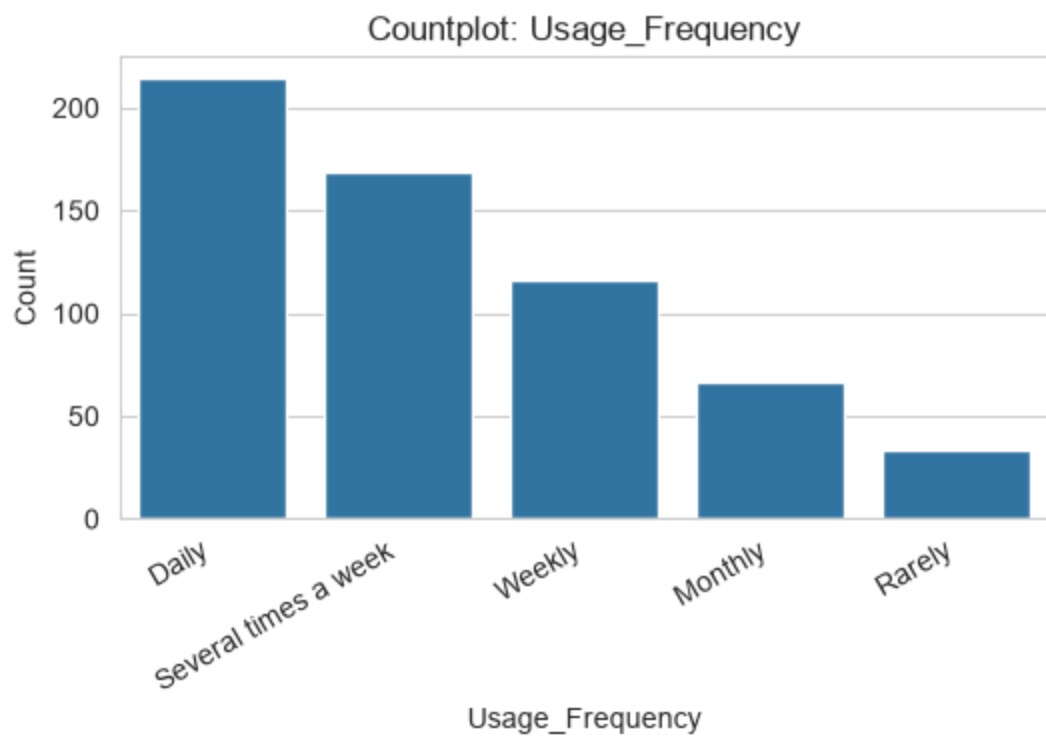
```
In [38]: #Countplots
```

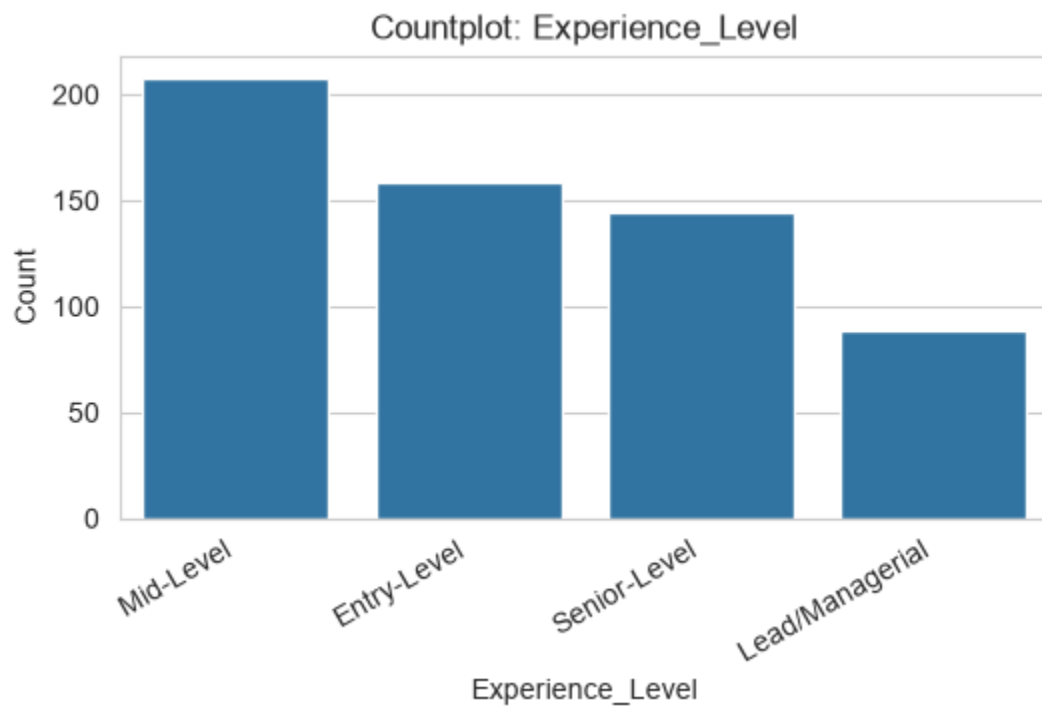
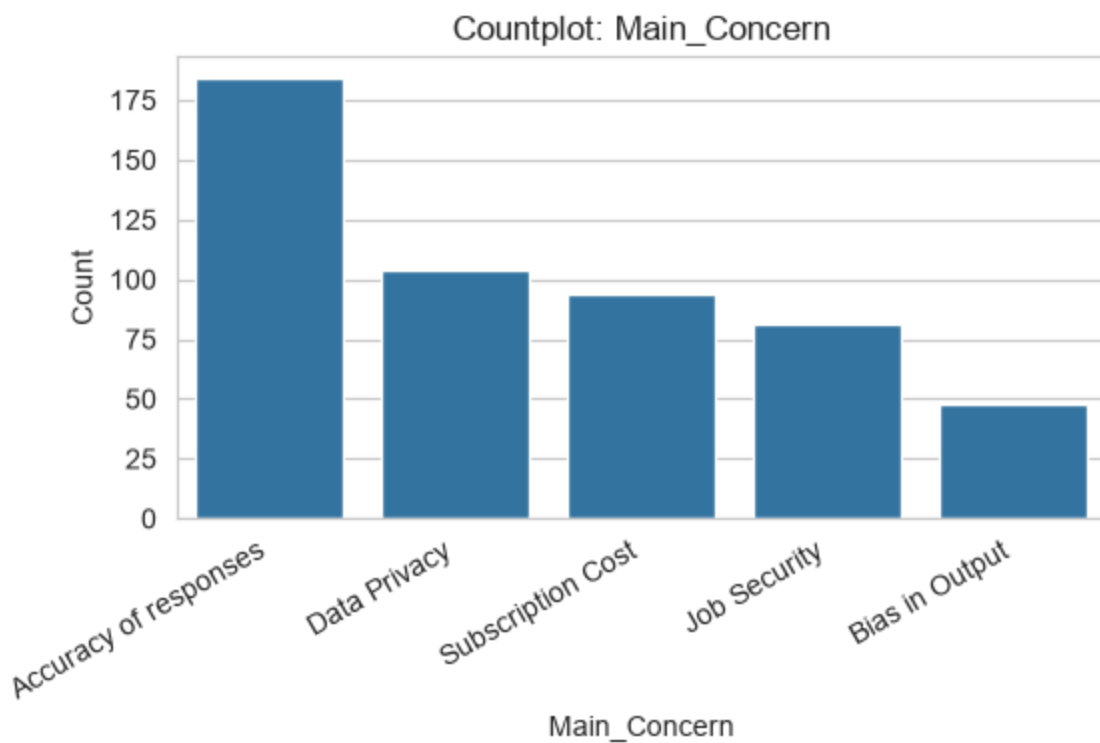
```
In [39]: top_level_cats = ["Primary_LLM_Tool", "Subscription_Type", "Usage_Frequency",
                          "Would_Recommend", "Main_Concern", "Experience_Level"]

for col in top_level_cats:
    plt.figure(figsize=(6, 3))
    order = df[col].value_counts().index
    sns.countplot(x=df[col], order=order)
```

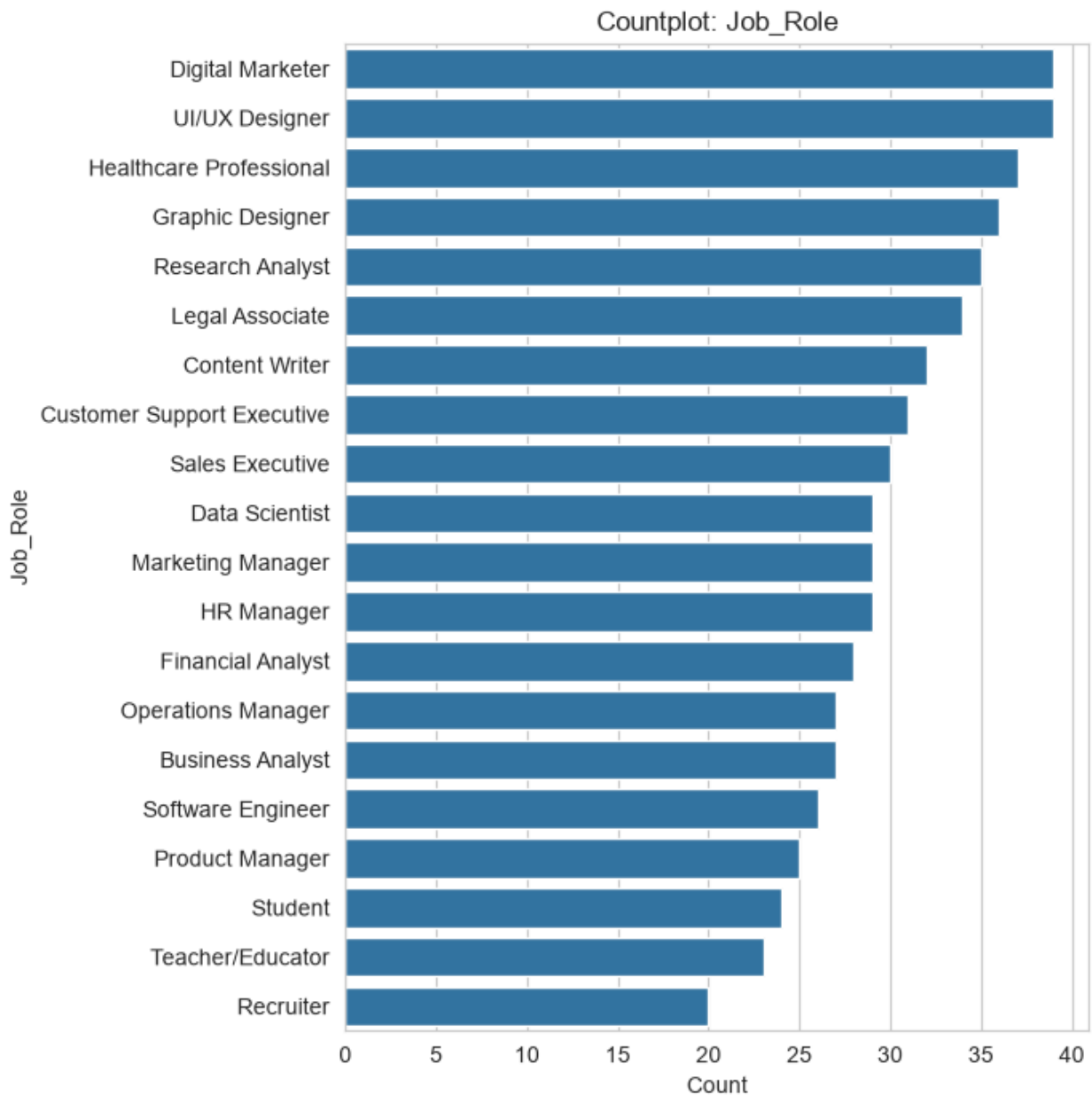
```
plt.title(f"Countplot: {col}")
plt.xlabel(col)
plt.ylabel("Count")
plt.xticks(rotation=30, ha="right")
plt.show()
```







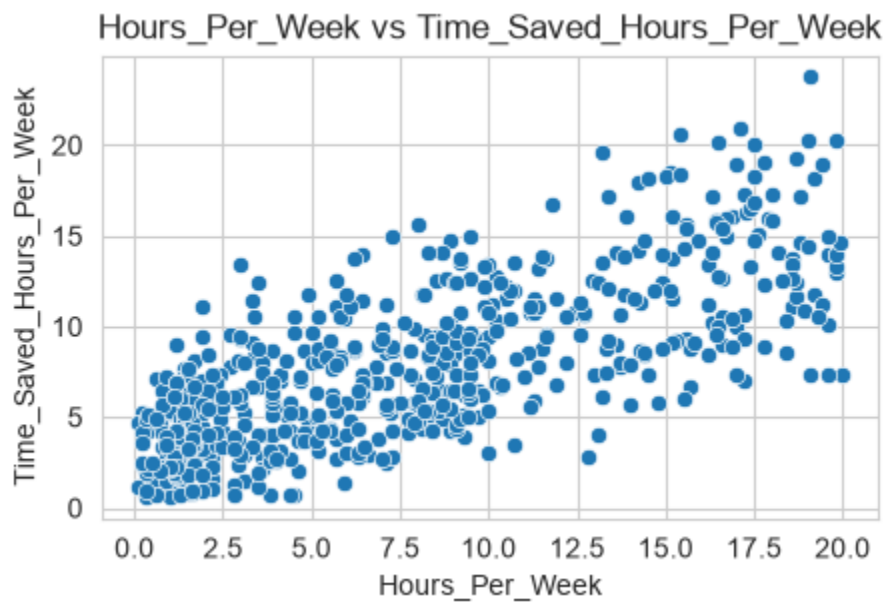
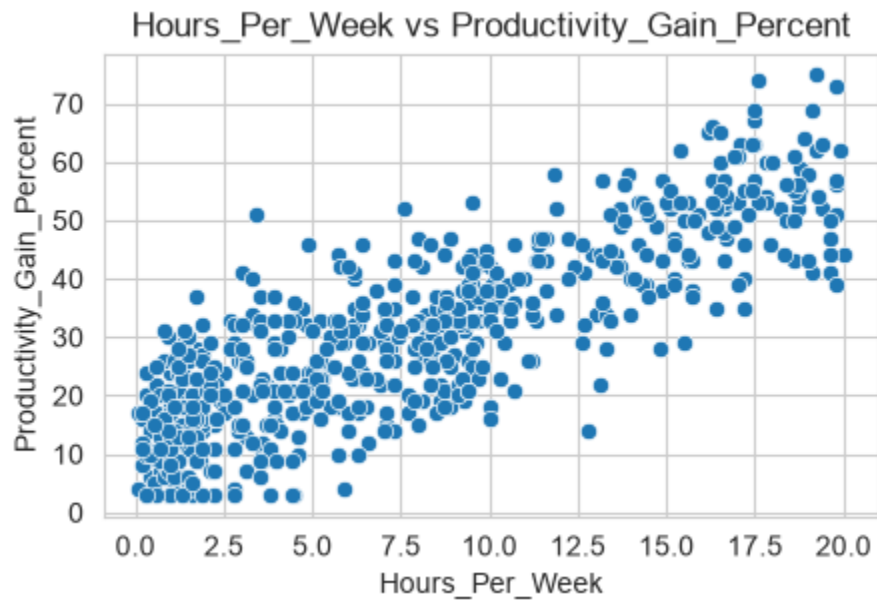
```
In [40]: # Job_Role has 20 categories
plt.figure(figsize=(6, 8))
order = df["Job_Role"].value_counts().index
sns.countplot(y=df["Job_Role"], order=order)
plt.title("Countplot: Job_Role")
plt.xlabel("Count")
plt.ylabel("Job_Role")
plt.show()
```



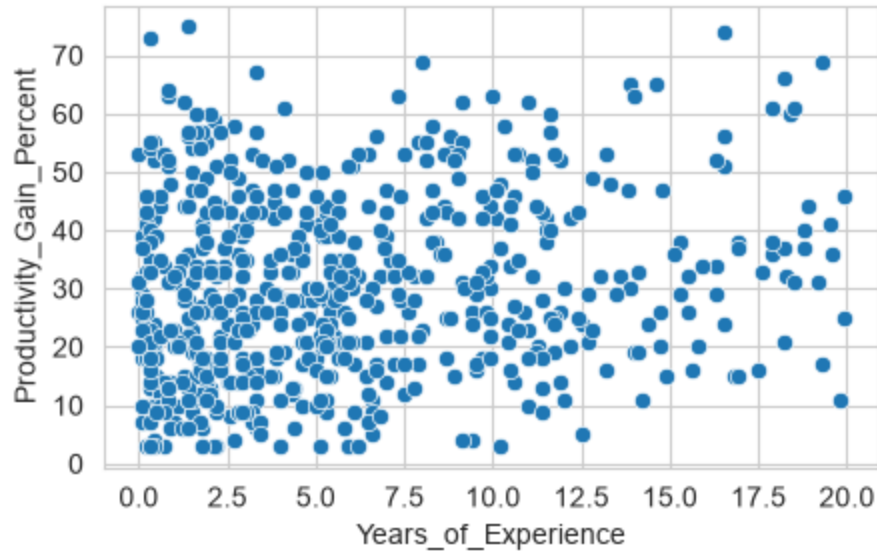
```
In [43]: # Scatter plots
```

```
In [44]: num_pairs = [  
    ("Hours_Per_Week", "Productivity_Gain_Percent"),  
    ("Hours_Per_Week", "Time_Saved_Hours_Per_Week"),  
    ("Years_of_Experience", "Productivity_Gain_Percent"),  
    ("Monthly_Cost_USD", "Overall_Satisfaction_Score_1to10"),  
    ("Accuracy_Satisfaction_Score_1to10", "Overall_Satisfaction_Score_1to10")  
]  
  
for x, y in num_pairs:  
    plt.figure(figsize=(5, 3))  
    sns.scatterplot(x=df[x], y=df[y])  
    plt.title(f"{x} vs {y}")  
    plt.xlabel(x)
```

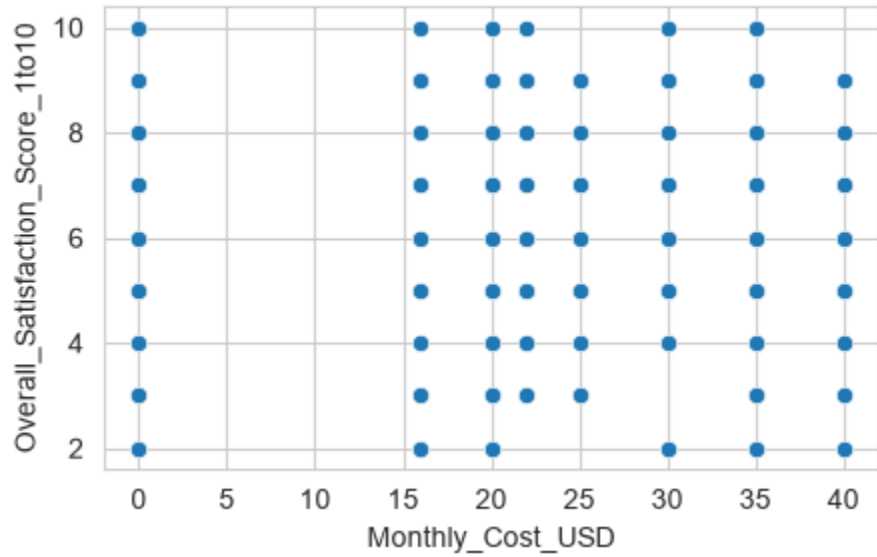
```
plt.ylabel(y)
plt.show()
```



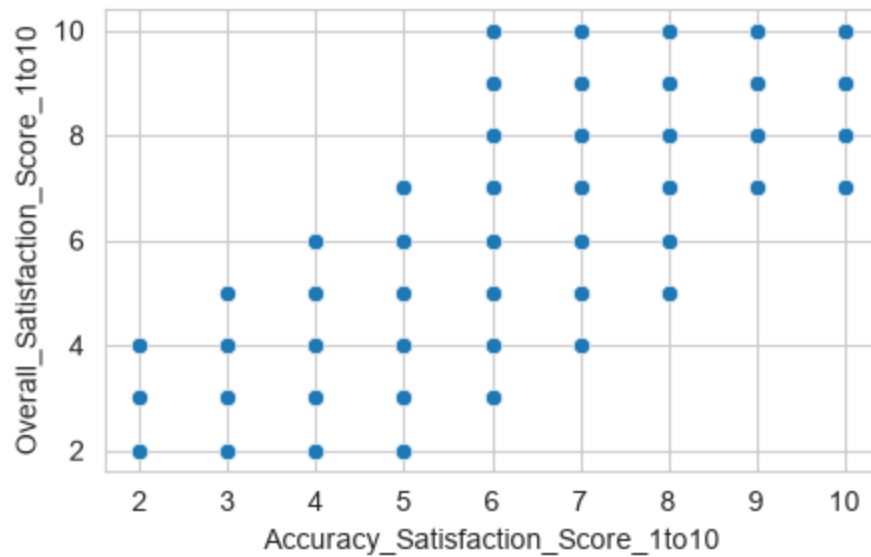
Years_of_Experience vs Productivity_Gain_Percent



Monthly_Cost_USD vs Overall_Satisfaction_Score_1to10



Accuracy_Satisfaction_Score_1to10 vs Overall_Satisfaction_Score_1to10



In [45]: `#Correlation matrix`

In [46]: `print("Correlation Matrix")
df[num_cols].corr()`

Correlation Matrix

Out[46]:

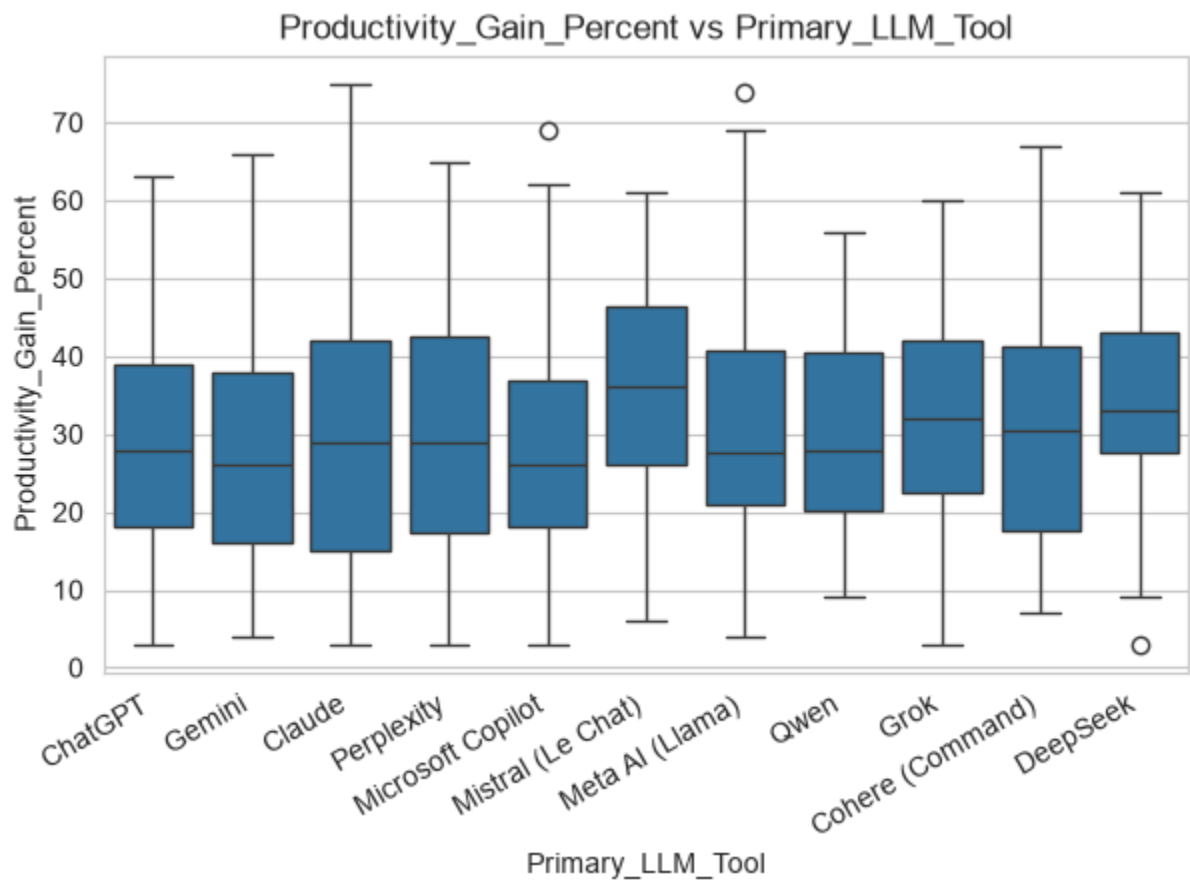
	Years_of_Experience	Monthly_Cost_USD	Hou
Years_of_Experience	1.000	0.022	
Monthly_Cost_USD	0.022	1.000	
Hours_Per_Week	0.018	-0.026	
Months_Since_Adoption	0.027	-0.013	
Tasks_Automated_Per_Week	0.030	-0.016	
Productivity_Gain_Percent	0.179	0.004	
Time_Saved_Hours_Per_Week	0.154	-0.017	
Accuracy_Satisfaction_Score_1to10	-0.014	-0.109	
Overall_Satisfaction_Score_1to10	-0.038	-0.111	
Trust_Level_1to10	-0.048	-0.110	

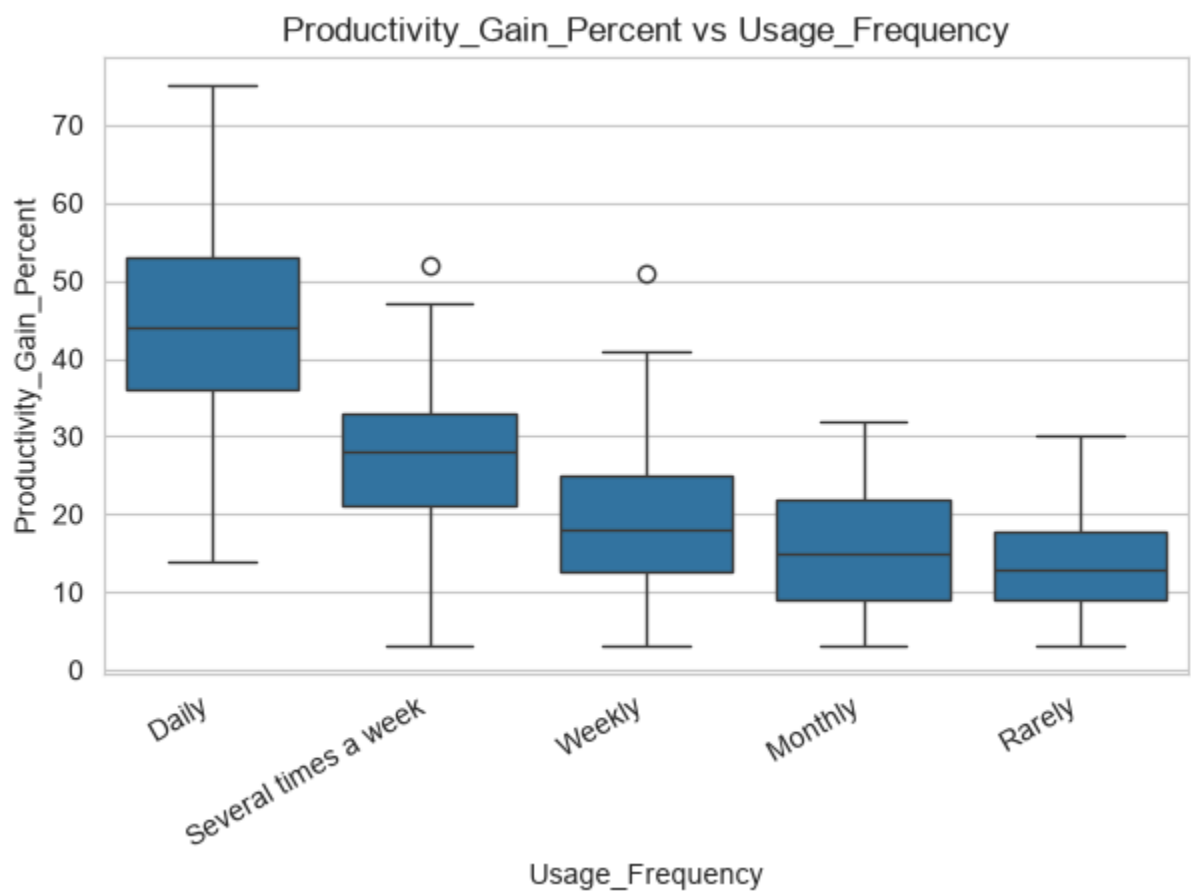
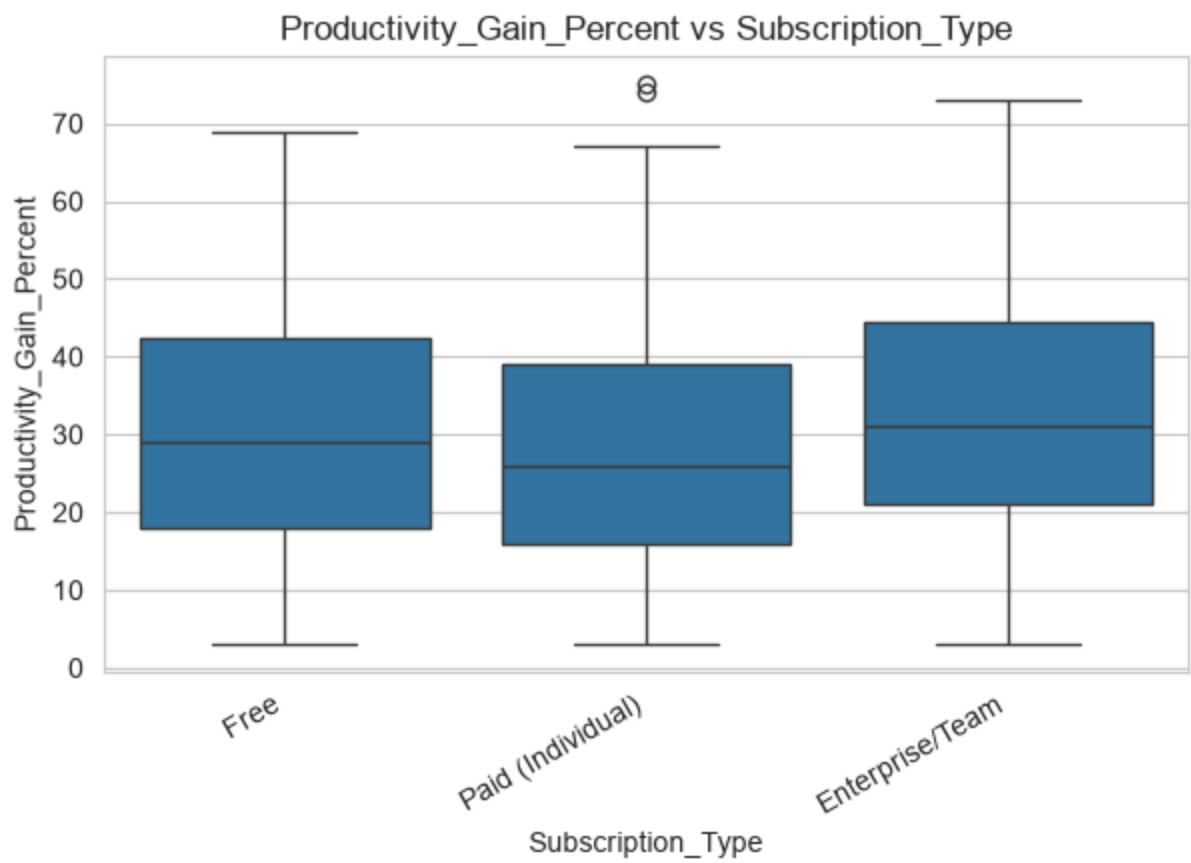
In [47]: `print("Pearson correlation with Productivity_Gain_Percent:")
for col in num_cols:
 if col != "Productivity_Gain_Percent":
 r, p = pearsonr(df[col], df["Productivity_Gain_Percent"])
 print(f"{col}: r={r:.3f}, p={p:.3g}")`

Pearson correlation with Productivity_Gain_Percent:
 Years_of_Experience: $r=0.179$, $p=9.79e-06$
 Monthly_Cost_USD: $r=0.004$, $p=0.916$
 Hours_Per_Week: $r=0.833$, $p=8.79e-156$
 Months_Since_Adoption: $r=0.022$, $p=0.585$
 Tasks_Automated_Per_Week: $r=0.788$, $p=4.82e-128$
 Time_Saved_Hours_Per_Week: $r=0.896$, $p=8.16e-213$
 Accuracy_Satisfaction_Score_1to10: $r=0.040$, $p=0.325$
 Overall_Satisfaction_Score_1to10: $r=0.030$, $p=0.456$
 Trust_Level_1to10: $r=0.045$, $p=0.275$

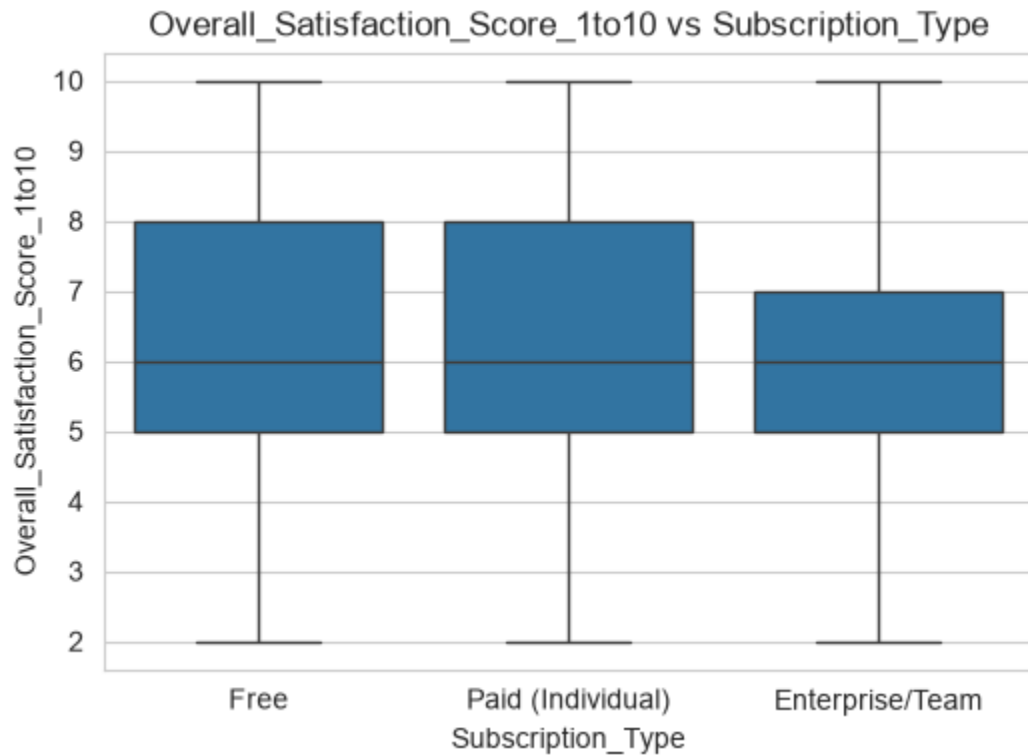
In [48]: `#Numerical vs Categorical`

```
In [49]: for cat in ["Primary_LLM_Tool", "Subscription_Type", "Usage_Frequency"]:
plt.figure(figsize=(7, 4))
order = df[cat].value_counts().index
sns.boxplot(x=df[cat], y=df["Productivity_Gain_Percent"], order=order)
plt.title(f"Productivity_Gain_Percent vs {cat}")
plt.xlabel(cat)
plt.ylabel("Productivity_Gain_Percent")
plt.xticks(rotation=30, ha="right")
plt.show()
```





```
In [50]: plt.figure(figsize=(6, 4))
sns.boxplot(x=df["Subscription_Type"], y=df["Overall_Satisfaction_Score_1to10"]
plt.title("Overall_Satisfaction_Score_1to10 vs Subscription_Type")
plt.show()
```



```
In [51]: #GROUPED STATS
```

```
In [52]: df[num_cols + ["Primary_LLM_Tool"]].groupby("Primary_LLM_Tool").mean().sort_val
         "Productivity_Gain_Percent", ascending=False
         )
```

Out[52]:

	Years_of_Experience	Monthly_Cost_USD	Hours_Per_Week	Mo
Primary_LLM_Tool				
Mistral (Le Chat)	7.729	15.516	9.245	
DeepSeek	4.556	13.625	8.669	
Grok	6.174	13.842	8.568	
Meta AI (Llama)	7.493	6.643	8.482	
Perplexity	6.065	16.278	7.778	
Cohere (Command)	5.556	14.889	8.194	
Claude	5.435	15.346	8.088	
ChatGPT	5.387	12.895	7.397	
Qwen	6.950	8.850	7.365	
Gemini	6.714	14.030	7.385	
Microsoft Copilot	5.387	14.415	6.260	

```
In [53]: for category in ["Subscription_Type", "Usage_Frequency", "Experience_Level"]:
          print(df[num_cols + [category]].groupby(category).mean())
          print("-"*50)
```

	Years_of_Experience	Monthly_Cost_USD	Hours_Per_Week \
Subscription_Type			
Enterprise/Team	6.199	31.757	8.299
Free	5.862	0.000	8.057
Paid (Individual)	5.936	20.335	6.910

	Months_Since_Adoption	Tasks_Automated_Per_Week \
Subscription_Type		
Enterprise/Team	23.234	10.694
Free	23.552	10.351
Paid (Individual)	23.217	8.909

	Productivity_Gain_Percent	Time_Saved_Hours_Per_Week \
Subscription_Type		
Enterprise/Team	32.369	8.268
Free	30.386	7.959
Paid (Individual)	28.343	7.208

	Accuracy_Satisfaction_Score_1to10 \
Subscription_Type	
Enterprise/Team	6.099
Free	6.575
Paid (Individual)	6.378

	Overall_Satisfaction_Score_1to10	Trust_Level_1to10
Subscription_Type		
Enterprise/Team	5.838	4.297
Free	6.402	4.919
Paid (Individual)	6.213	4.774

	Years_of_Experience	Monthly_Cost_USD	Hours_Per_Week \
Usage_Frequency			
Daily	6.010	13.304	14.206
Monthly	5.681	16.149	1.290
Rarely	5.297	13.941	0.526
Several times a week	6.227	13.018	6.882
Weekly	5.797	13.784	2.498

	Months_Since_Adoption	Tasks_Automated_Per_Week \
Usage_Frequency		
Daily	24.210	18.098
Monthly	21.269	2.313
Rarely	23.559	2.000
Several times a week	22.213	8.260
Weekly	24.638	3.664

	Productivity_Gain_Percent	Time_Saved_Hours_Per_Week \
Usage_Frequency		
Daily	44.593	11.555
Monthly	15.761	4.145
Rarely	13.676	3.297
Several times a week	27.763	7.156
Weekly	19.190	4.871

	Accuracy_Satisfaction_Score_1to10 \	
Usage_Frequency		
Daily	6.533	
Monthly	6.000	
Rarely	5.971	
Several times a week	6.426	
Weekly	6.534	

	Overall_Satisfaction_Score_1to10	Trust_Level_1to10
Usage_Frequency		
Daily	6.336	4.879
Monthly	5.925	4.552
Rarely	5.618	4.176
Several times a week	6.195	4.734
Weekly	6.414	4.810

	Years_of_Experience	Monthly_Cost_USD	Hours_Per_Week \
Experience_Level			
Entry-Level	0.969	14.403	7.558
Lead/Managerial	14.947	14.584	8.056
Mid-Level	3.965	12.620	7.625
Senior-Level	8.767	13.812	7.587

	Months_Since_Adoption	Tasks_Automated_Per_Week \
Experience_Level		
Entry-Level	22.428	9.616
Lead/Managerial	23.157	10.393
Mid-Level	23.798	9.812
Senior-Level	23.903	9.875

	Productivity_Gain_Percent	Time_Saved_Hours_Per_Week \
Experience_Level		
Entry-Level	27.572	7.031
Lead/Managerial	35.685	9.022
Mid-Level	28.101	7.312
Senior-Level	31.785	8.300

	Accuracy_Satisfaction_Score_1to10 \	
Experience_Level		
Entry-Level	6.396	
Lead/Managerial	6.539	
Mid-Level	6.452	
Senior-Level	6.292	

	Overall_Satisfaction_Score_1to10	Trust_Level_1to10
Experience_Level		
Entry-Level	6.252	4.736
Lead/Managerial	6.270	4.742
Mid-Level	6.312	4.880
Senior-Level	6.042	4.576

In [54]: #Categorical vs Categorical

```
In [55]: print("Crosstab: Primary_LLM_Tool vs Subscription_Type")
pd.crosstab(df["Primary_LLM_Tool"], df["Subscription_Type"])
```

Crosstab: Primary_LLM_Tool vs Subscription_Type

```
Out[55]: Subscription_Type Enterprise/Team Free Paid (Individual)
```

Primary_LLM_Tool			
ChatGPT	28	82	71
Claude	20	33	28
Cohere (Command)	3	7	8
DeepSeek	3	6	7
Gemini	21	40	38
Grok	2	8	9
Meta AI (Llama)	1	19	8
Microsoft Copilot	10	22	21
Mistral (Le Chat)	6	10	15
Perplexity	15	20	19
Qwen	2	12	6

```
In [56]: print("Crosstab: Job_Role vs Primary_LLM_Tool (top 5 roles)")
top_roles = df["Job_Role"].value_counts().head(5).index
pd.crosstab(df[df["Job_Role"].isin(top_roles)]["Job_Role"], df["Primary_LLM_Tool"])
```

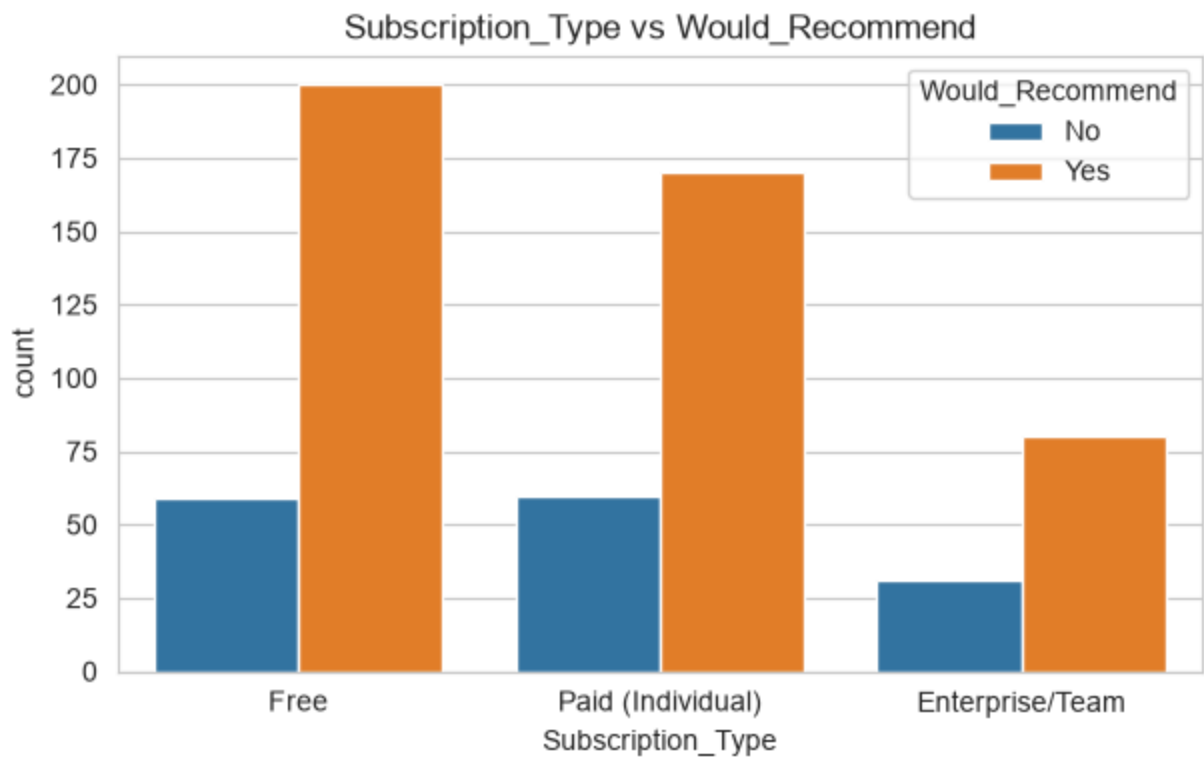
Crosstab: Job_Role vs Primary_LLM_Tool (top 5 roles)

```
Out[56]: Primary_LLM_Tool ChatGPT Claude Cohere (Command) DeepSeek Gemini Grok Meta AI (Llama)
```

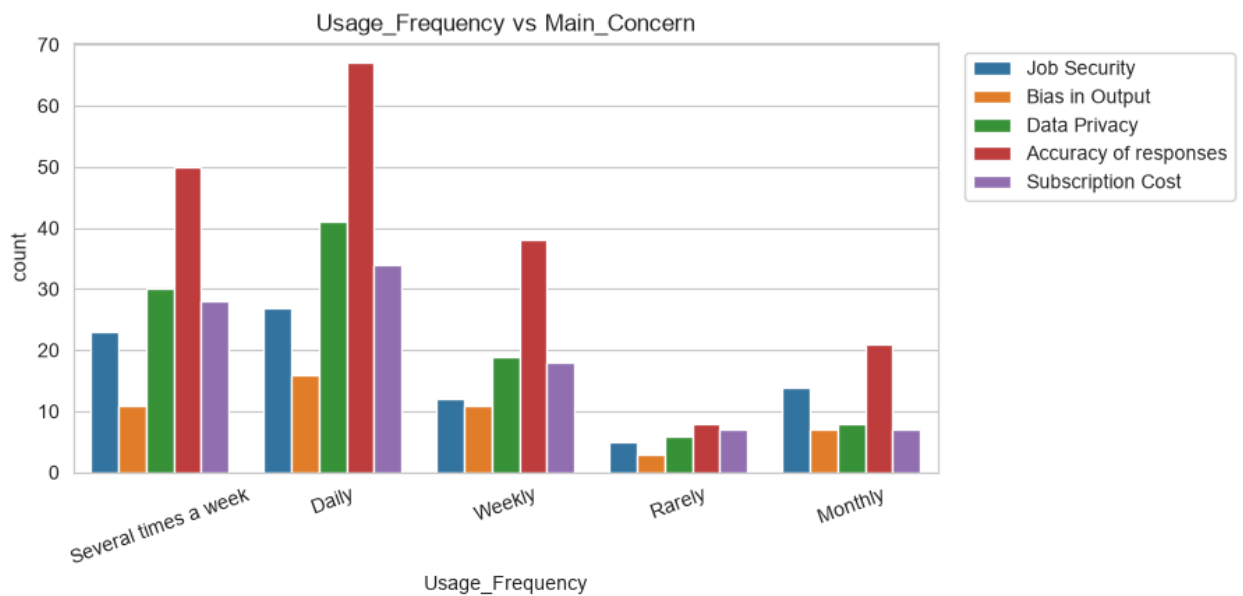
Job_Role						
Digital Marketer	13	5	0	0	4	2
Graphic Designer	9	3	1	1	7	3
Healthcare Professional	12	7	4	0	5	0
Research Analyst	4	5	3	1	5	0
UI/UX Designer	8	3	2	1	8	0

```
In [57]: #Countplot (grouped)
```

```
In [58]: plt.figure(figsize=(7, 4))
sns.countplot(x=df["Subscription_Type"], hue=df["Would_Recommend"])
plt.title("Subscription_Type vs Would_Recommend")
plt.show()
```



```
In [59]: plt.figure(figsize=(8, 4))
sns.countplot(x=df["Usage_Frequency"], hue=df["Main_Concern"])
plt.title("Usage_Frequency vs Main_Concern")
plt.xticks(rotation=20)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.show()
```

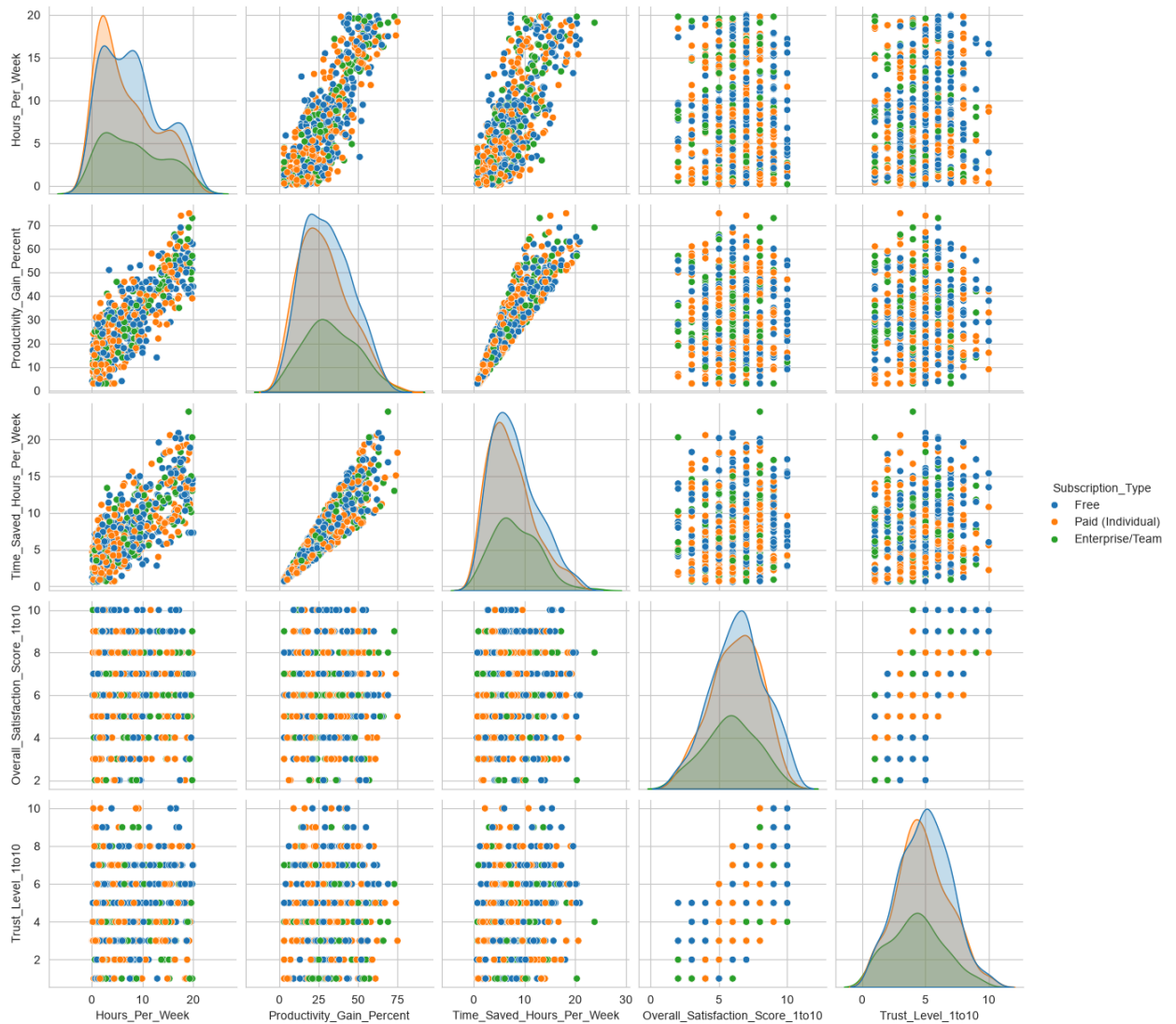


```
In [60]: #Pair plot
```

```
In [62]: key_num = ["Hours_Per_Week", "Productivity_Gain_Percent", "Time_Saved_Hours_Percent", "Overall_Satisfaction_Score_1to10", "Trust_Level_1to10"]
```

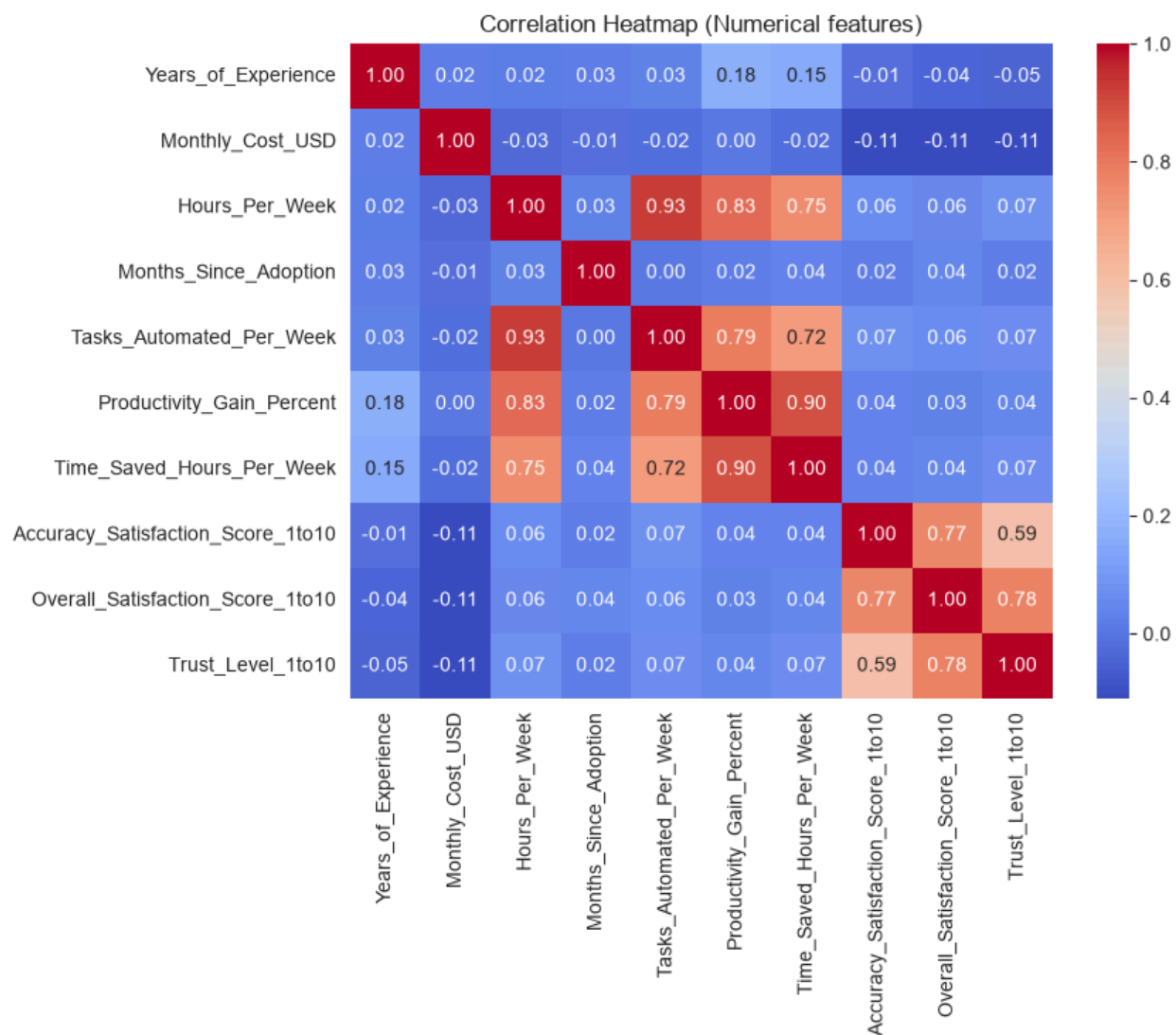
```
In [63]: pairplot_cols = key_num + ["Subscription_Type"]
df_pair = df[pairplot_cols].dropna()

sns.pairplot(df_pair, hue="Subscription_Type")
plt.show()
```



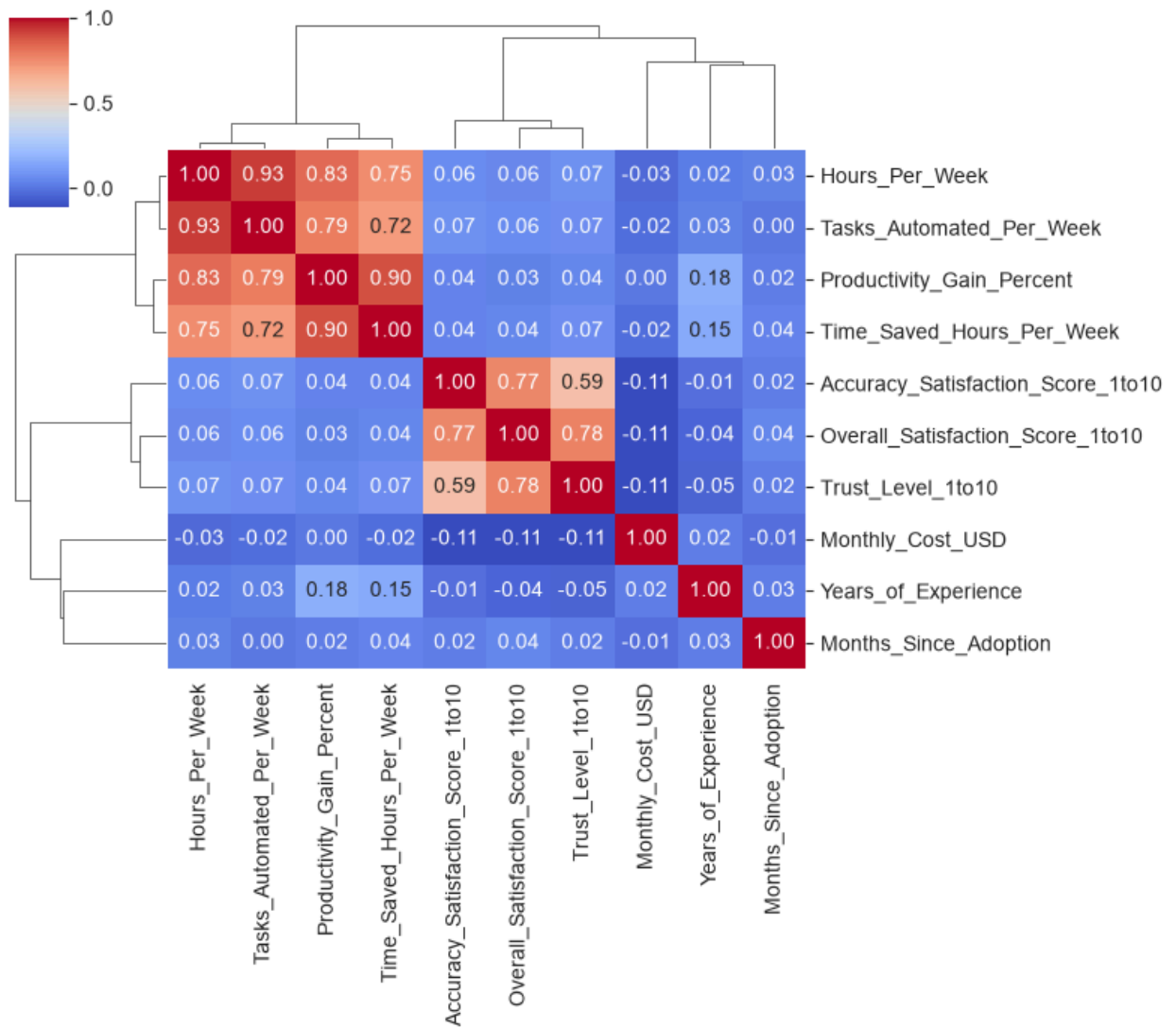
```
In [64]: #Correlation heatmap
```

```
In [65]: plt.figure(figsize=(8, 6))
sns.heatmap(df[num_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap (Numerical features)")
plt.show()
```

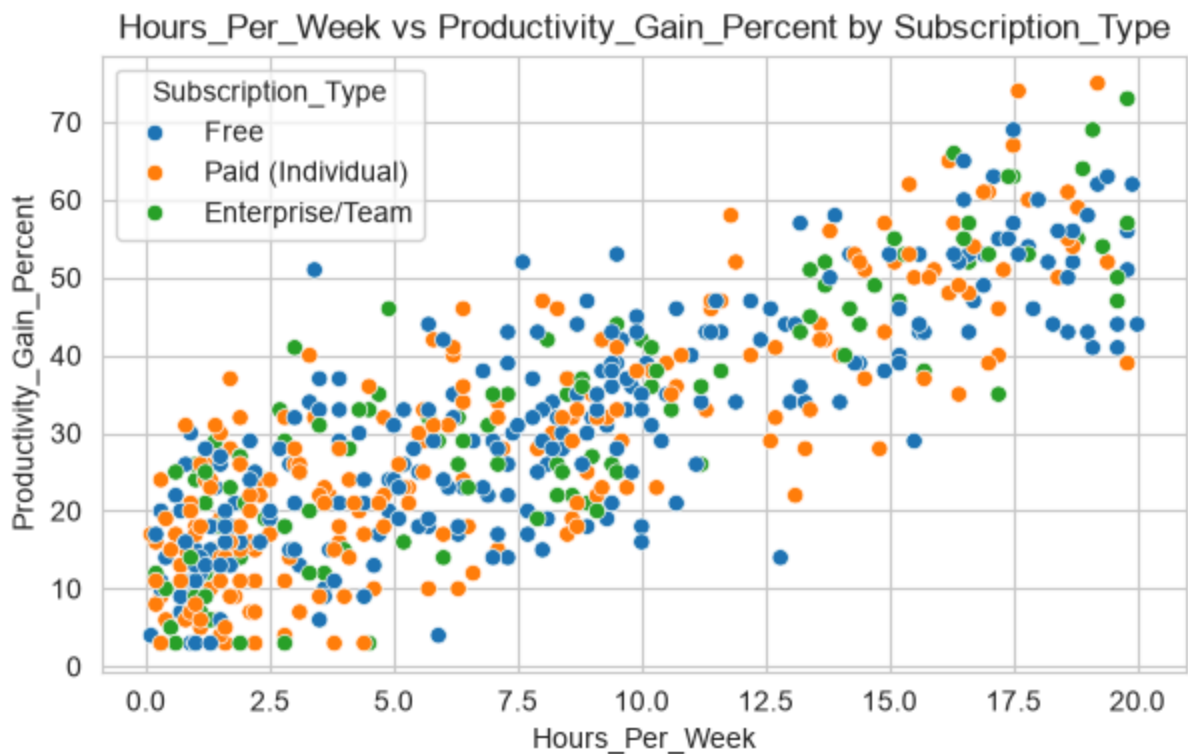


```
In [66]: #Cluster map
```

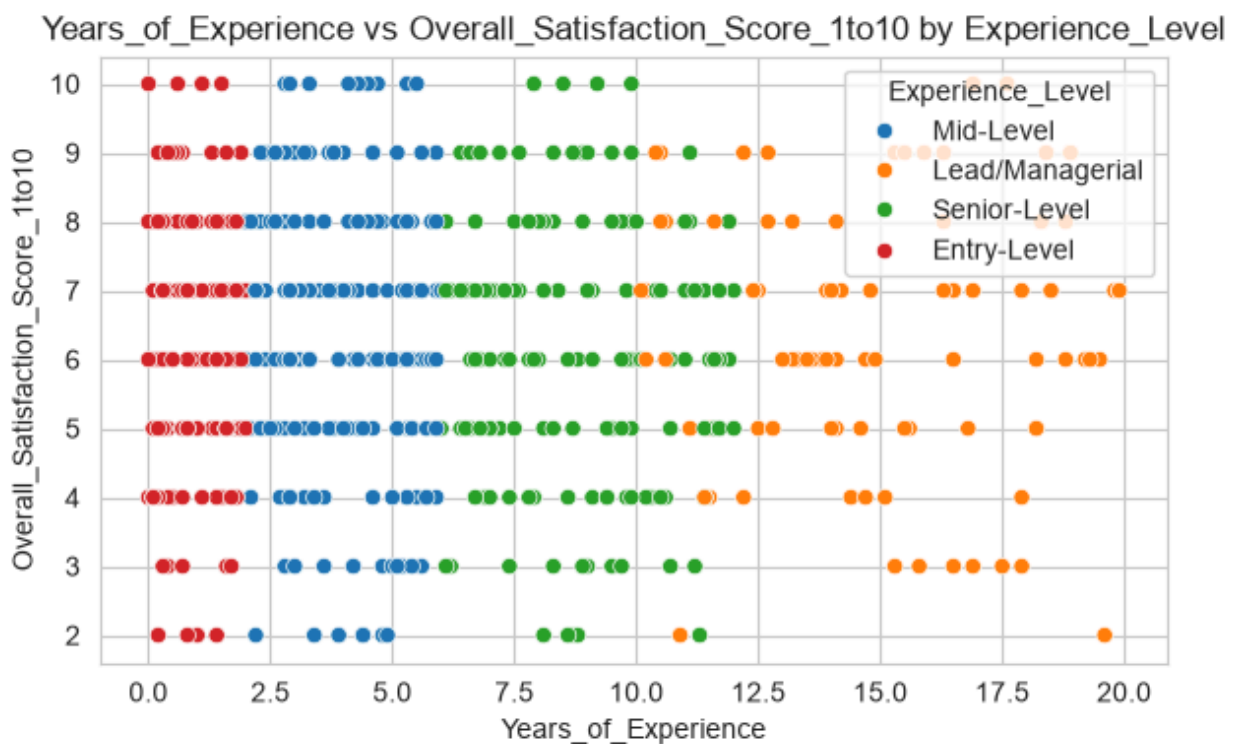
```
In [67]: sns.clustermap(df[num_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f", fi
plt.show()
```



```
In [69]: plt.figure(figsize=(7, 4))
sns.scatterplot(x=df["Hours_Per_Week"], y=df["Productivity_Gain_Percent"], hue=
plt.title("Hours_Per_Week vs Productivity_Gain_Percent by Subscription_Type")
plt.show()
```



```
In [70]: plt.figure(figsize=(7, 4))
sns.scatterplot(x=df["Years_of_Experience"], y=df["Overall_Satisfaction_Score_1to10"])
plt.title("Years_of_Experience vs Overall_Satisfaction_Score_1to10 by Experience_Level")
plt.show()
```



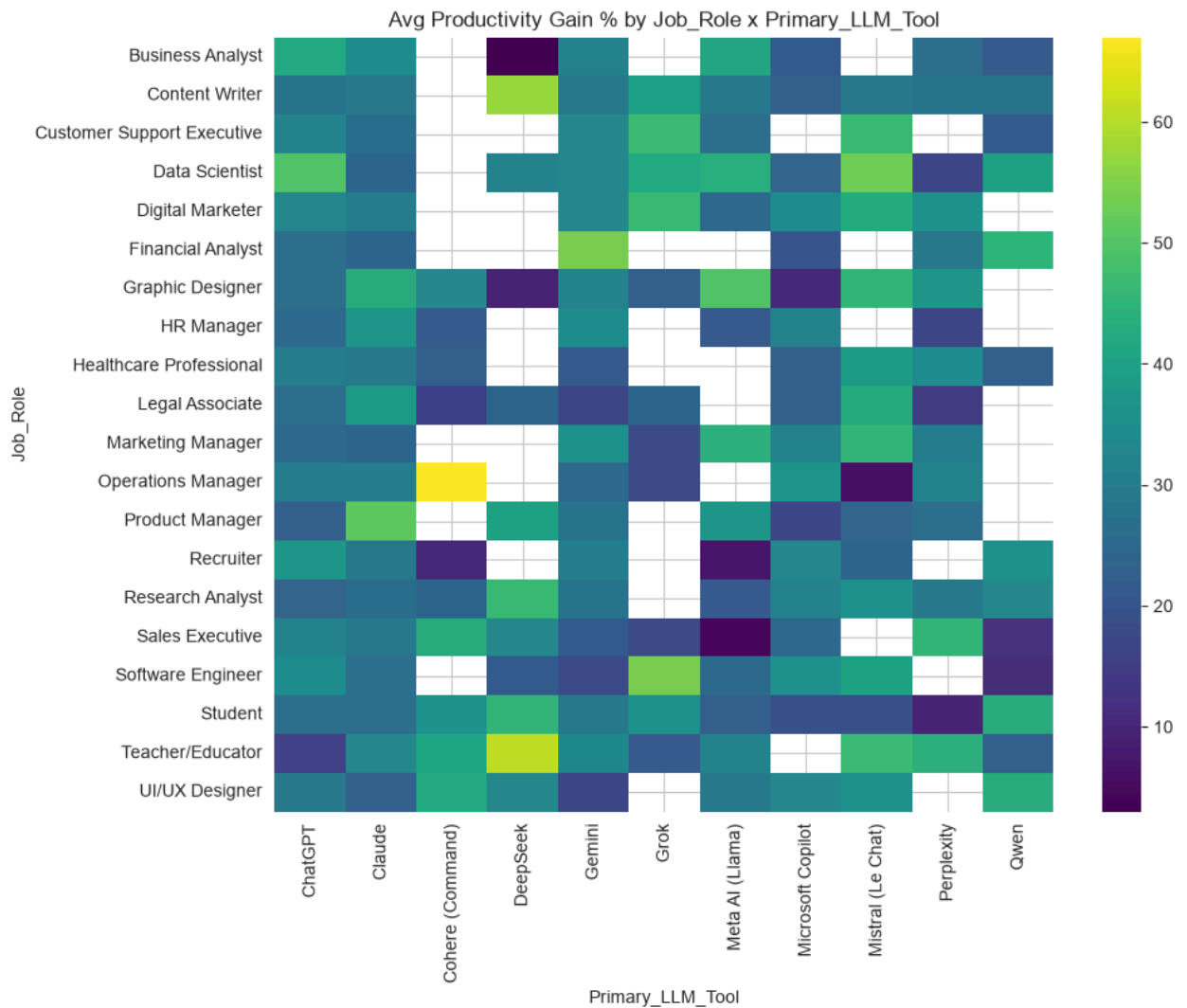
```
In [71]: # average productivity gain by role and top LLM tool (pivot heatmap)
```

```

pivot = df.pivot_table(values="Productivity_Gain_Percent", index="Job_Role",
                        columns="Primary_LLM_Tool", aggfunc="mean")

plt.figure(figsize=(10, 8))
sns.heatmap(pivot, annot=False, cmap="viridis")
plt.title("Avg Productivity Gain % by Job_Role x Primary_LLM_Tool")
plt.show()

```



In [72]: *#IQR method*

```

def iqr_bounds(series, factor=1.5):
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - factor * iqr
    upper_bound = q3 + factor * iqr
    return lower_bound, upper_bound, iqr

```

```

for col in num_cols:
    col_clean = df[col].dropna()
    lower, upper, iqr = iqr_bounds(col_clean)

```

```

outliers_mask = (col_clean < lower) | (col_clean > upper)
num_outliers = outliers_mask.sum()
total = col_clean.shape[0]

print(f"\nIQR method for {col}:")
print(f" Lower bound: {lower:.2f}, Upper bound: {upper:.2f}, IQR: {iqr:.2f}")
print(f" Outliers: {num_outliers} / {total} ({num_outliers/total*100:.2f})%")

```

IQR method for Years_of_Experience:

Lower bound: -8.79, Upper bound: 19.71, IQR: 7.12
 Outliers: 3 / 600 (0.50%)

IQR method for Monthly_Cost_USD:

Lower bound: -33.00, Upper bound: 55.00, IQR: 22.00
 Outliers: 0 / 600 (0.00%)

IQR method for Hours_Per_Week:

Lower bound: -11.79, Upper bound: 25.51, IQR: 9.32
 Outliers: 0 / 600 (0.00%)

IQR method for Months_Since_Adoption:

Lower bound: -16.00, Upper bound: 64.00, IQR: 20.00
 Outliers: 0 / 600 (0.00%)

IQR method for Tasks_Automated_Per_Week:

Lower bound: -16.50, Upper bound: 35.50, IQR: 13.00
 Outliers: 0 / 600 (0.00%)

IQR method for Productivity_Gain_Percent:

Lower bound: -16.50, Upper bound: 75.50, IQR: 23.00
 Outliers: 0 / 600 (0.00%)

IQR method for Time_Saved_Hours_Per_Week:

Lower bound: -5.29, Upper bound: 20.01, IQR: 6.33
 Outliers: 7 / 600 (1.17%)

IQR method for Accuracy_Satisfaction_Score_1to10:

Lower bound: 2.00, Upper bound: 10.00, IQR: 2.00
 Outliers: 0 / 600 (0.00%)

IQR method for Overall_Satisfaction_Score_1to10:

Lower bound: 0.50, Upper bound: 12.50, IQR: 3.00
 Outliers: 0 / 600 (0.00%)

IQR method for Trust_Level_1to10:

Lower bound: -1.50, Upper bound: 10.50, IQR: 3.00
 Outliers: 0 / 600 (0.00%)

In [75]: *#Z-score method*

```

In [76]: z_threshold = 3.0

for col in num_cols:
    col_clean = df[col].dropna()

```

```

mean = col_clean.mean()
std = col_clean.std()

z_scores = (col_clean - mean) / std
outliers_mask = np.abs(z_scores) > z_threshold
num_outliers = outliers_mask.sum()
total = col_clean.shape[0]

print(f"{col}: {num_outliers} / {total} outliers (|Z| > {z_threshold})")

```

```

Years_of_Experience: 0 / 600 outliers (|Z| > 3.0)
Monthly_Cost_USD: 0 / 600 outliers (|Z| > 3.0)
Hours_Per_Week: 0 / 600 outliers (|Z| > 3.0)
Months_Since_Adoption: 0 / 600 outliers (|Z| > 3.0)
Tasks_Automated_Per_Week: 0 / 600 outliers (|Z| > 3.0)
Productivity_Gain_Percent: 0 / 600 outliers (|Z| > 3.0)
Time_Saved_Hours_Per_Week: 1 / 600 outliers (|Z| > 3.0)
Accuracy_Satisfaction_Score_1to10: 0 / 600 outliers (|Z| > 3.0)
Overall_Satisfaction_Score_1to10: 0 / 600 outliers (|Z| > 3.0)
Trust_Level_1to10: 0 / 600 outliers (|Z| > 3.0)

```

In [77]: *#Stats comparison after removing outliers (Productivity_Gain_Percent)*

```

In [78]: pg_clean = df["Productivity_Gain_Percent"].dropna()
pg_lower, pg_upper, pg_iqr = iqr_bounds(pg_clean)
pg_inliers = pg_clean[(pg_clean >= pg_lower) & (pg_clean <= pg_upper)]

print("Productivity_Gain_Percent stats WITH outliers:")
print(pg_clean.describe())
print("\nProductivity_Gain_Percent stats WITHOUT outliers:")
print(pg_inliers.describe())

```

```

Productivity_Gain_Percent stats WITH outliers:
count    600.000
mean      29.970
std       15.586
min        3.000
25%       18.000
50%       28.000
75%       41.000
max       75.000
Name: Productivity_Gain_Percent, dtype: float64

```

```

Productivity_Gain_Percent stats WITHOUT outliers:
count    600.000
mean      29.970
std       15.586
min        3.000
25%       18.000
50%       28.000
75%       41.000
max       75.000
Name: Productivity_Gain_Percent, dtype: float64

```